



UNIVERSITÀ DI PISA

MASTER OF SCIENCE IN COMPUTER ENGINEERING
LARGE-SCALE AND MULTI-STRUCTURED
DATABASES

BookSphere

Books Review Social Network

Authors:

Danilo PARISI

Matteo IORI

Matteo BILLI CIANI

A.Y. 2025–2026

Contents

1	Introduction	5
1.1	Project Overview	5
1.2	Motivation and Key Features	5
2	System Design	7
2.1	System Architecture	7
2.1.1	Architectural Layers	7
2.2	Requirements Analysis	8
2.2.1	Main Actors	8
2.2.2	Functional Requirements	8
2.2.3	Non-Functional Requirements	11
2.3	Expected Workload Distribution	11
2.3.1	Workload Profile (OLTP vs. OLAP)	11
2.4	UML Class Diagram	12
2.5	User Interface Concept and Mock-ups	13
2.5.1	Unregistered User Experience	13
2.5.2	Registered User Experience	14
2.5.3	Administrator Experience	16
3	Data Modeling	18
3.1	Data Source and Volume	18
3.2	Dataset Preprocessing	19
3.2.1	ETL Pipeline	19
3.2.2	Deployment Scripts	19
3.3	Document Store: MongoDB	20
3.3.1	Collections Overview	20
3.3.2	Schema Design (JSON)	20

3.3.3	Design Rationale	24
3.4	Graph Store: Neo4j	24
3.4.1	Nodes and Labels	25
3.4.2	Relationships	25
3.4.3	Graph Schema Visualization	25
3.4.4	Design Rationale	26
4	Databases Choices and Distributed Architecture	27
4.1	Persistence Strategy and Data Consistency	27
4.1.1	Polyglot Persistence Architecture	27
4.1.2	Consistency Strategies	27
4.1.3	System Resilience	28
4.1.4	Synchronization Matrix	28
4.2	Indexes Justification and Validation	30
4.2.1	MongoDB Indexes	30
4.2.2	Neo4j Indexes	32
4.3	Virtual Machine Organization and Replica Sets	33
4.3.1	Document Database Deployment	33
4.3.2	Graph Database Deployment	34
4.4	Sharding Strategy	34
4.5	CAP Theorem Considerations	35
5	Implementation	37
5.1	Technologies and Frameworks	37
5.2	Data Deletion Strategies and Lifecycle Management	37
5.2.1	User Unregistration: The Hybrid Anonymization Approach	38
5.2.2	Review Deletion: Hard Delete with Cascade Consistency	38
5.2.3	Catalog Lifecycle (Book/Author): Archival Strategy	39
5.3	Directory Structure and Modules	39
5.3.1	Config	39
5.3.2	Controllers	40
5.3.3	Model	42
5.3.4	DTO (Data Transfer Objects)	44
5.3.5	Mappers	45
5.3.6	Repositories	47

5.3.7	Services and Async	50
5.3.8	Utils	56
5.3.9	Validation	57
5.3.10	Exceptions	58
5.4	MongoDB Relevant queries	59
5.4.1	Ranking	59
5.4.2	Revaluation	61
5.4.3	Yearly Wrapped	62
5.5	Neo4j Relevant Queries	63
5.5.1	User Recommendations	64
5.5.2	Internationality Index	64
5.5.3	Genre Influencer	65
6	System Testing and Performance Evaluation	66
6.1	Introduction	66
6.2	Java Integration Testing with JUnit 5	66
6.2.1	Key Testing Principles	66
6.2.2	Test Coverage by Access Tier	67
6.2.3	Cross-Database Consistency Validation	67
6.3	Performance Testing with Gatling	68
6.3.1	Test Results Summary	68
6.3.2	BasicLoadTest - Baseline Performance	68
6.3.3	StressTest - Capacity Validation	69
6.3.4	SpikeTest - Traffic Surge Resilience	69
6.3.5	EnduranceTest - Long-Term Stability	69
6.3.6	DatabaseIntensiveTest - Heavy Query Validation	69
6.4	End-to-End API Testing with Postman	70
6.4.1	Storytelling with Analytics Comparison	70
6.4.2	Complementary Live Demo Collection	73
7	API Documentation	75
7.1	API Architecture	75
7.2	Authentication Mechanism	75
7.3	Key Endpoints Overview	77
7.3.1	Open Endpoints	77

7.3.2	Registered User Endpoints	78
7.3.3	Admin Endpoints	79
7.4	API Examples	79
7.4.1	Example 1: User Registration and First Review	79
7.4.2	Example 2: Analytics Query	81
7.5	Error Handling	81
7.6	Security	82
A	AI Tools Usage	83
A.1	Purpose	83
A.2	Prompts and Tasks Performed	84
A.3	Critical Evaluation	84
A.3.1	Code Corrections and Architectural Refinements	85
A.3.2	Validation Against Course Requirements	85
A.4	Ethical Considerations	85
	Bibliography	87

Chapter 1

Introduction

Welcome to **BookSphere**, the ultimate social platform for book lovers designed to help you organize your reading life and connect with a global community of readers like you.

1.1 Project Overview

Imagine a place where your personal library is always at your fingertips, curated exactly how you live it—sorting your journey from the books you dream of reading, to the ones currently on your nightstand, to the stories that have already shaped you.

But a library should not be a static place. BookSphere allows you to build a circle of literary companions by following friends and fellow readers. Your feed transforms into a living book club, where you can track what your network is devouring in real-time, share reactions, and draw inspiration from the shelves of those you know best.

1.2 Motivation and Key Features

Beyond your personal circle, the motivation behind BookSphere is to connect users to a pulse of like-minded individuals, filtering out the noise of generic popularity. The platform introduces unique features designed to reward quality engagement:

- **Real Influencers:** We surface voices trusted not for their volume, but for the quality of their insight identifies, consistency and knowledge of a genre. Expert reviewers identified by the quality of their engagement rather than just follower count.
- **Revaluation Index:** Through data analysis, we track which books start with a negative public perception and then, over time, get re-evaluated for the better.
- **Trending Books:** Surfacing the stories that are sparking conversation right now, borne on the wave of the most recent reader feedback

- **Internationality Index:** We visualize how a story travels across borders, showing you that the book in your hands is being turned by another reader in Tokyo, New York, or Rome at this very moment.
- **Yearly Wrapped:** Ultimately, BookSphere is about your story. As you interact and rate, the platform aggregates your history to generate a personalized statistical summary at the end of the year—visualizing your top authors, favorite genres, and reading milestones.

Chapter 2

System Design

This chapter outlines the architectural blueprint of **BookSphere**. We begin by describing the high-level system design and technology stack, followed by a detailed analysis of functional and non-functional requirements, and concluding with the visual interface design.

2.1 System Architecture

The BookSphere platform is built upon a robust **3-Tier Architecture**, designed to ensure separation of concerns, scalability, and maintainability.

2.1.1 Architectural Layers

- **Presentation Layer (Frontend):** The client-side application is designed to be responsive and intuitive. It communicates with the backend via RESTful APIs, handling user interactions and data visualization (e.g., rendering the social graph or library shelves). While not implemented in this project due to the focus on API-based architecture, the mock-ups in Section 2.5 demonstrate the intended user experience.
- **Logic Layer (Backend):** The core of the system is developed using **Java 17** with the **Spring Boot 3.x** framework. This layer handles business logic, security (Authentication/Authorization via JWT), and acts as an orchestrator between the two different databases. We utilize **Lombok** to reduce boilerplate code, **MapStruct** for DTO mapping, and **Spring Data** modules for database integration. The architecture is detailed in Chapter 5.
- **Data Layer (Polyglot Persistence):** The system adopts a hybrid storage strategy:
 - **MongoDB:** Acts as the primary document store for Books, User Profiles, detailed Reviews, and aggregated statistics. Deployed with a 3-node replica set configuration for high availability.

- **Neo4j:** Serves as the high-performance graph engine to manage social connections (*FOLLOWS*) and interaction patterns (*LIKES*, *POSTED*) for the recommendation algorithms and social analytics.

2.2 Requirements Analysis

2.2.1 Main Actors

The system serves three main categories of users, each with distinct privileges and interaction capabilities:

- **Administrator:** The super-user responsible for platform management. Their tasks range from maintaining the consistency of the catalog (adding/updating books and authors) to community moderation (deleting content and banning users).
- **Registered User:** The primary target audience of the platform. These users can fully interact with the system by creating social connections, writing reviews, curating their library, and generating personal analytics.
- **Unregistered User (Guest):** Users who access the platform in "*read-only*" mode. They can explore the catalog, view public rankings and reviews, but cannot perform active interactions (rating, following, posting).

2.2.2 Functional Requirements

The functional requirements define the specific behaviors and functions of the system, categorized by the actor scope.

General & System Requirements

These requirements apply to the core logic of the platform and are available to all users (including Guests) or performed automatically by the system background processes.

Search & Discovery

1. The System must permit users to search for books and authors using different filters (genre, name).
2. The System must allow users to search for other profiles and view their reviews.
3. The System must enable any user to read reviews associated with a book.

System Analytics & Computation

4. The System must compute a metric representing an author's versatility based on the variety of genres covered in their works.
5. The System must compute a "How Far a Book Travels" index, measuring the geospatial distribution of a book's readers across the globe.
6. The System must generate various rankings, such as top books or authors, filtered by year or genre.
7. The System must calculate and display the average rating for authors and books for every year.
8. The System must identify trending books based on the volume of reviews and ratings in the current month.
9. The System must automatically identify "Influencers" within the application based on engagement quality metrics.

Access

11. The System must enable an Unregistered User to create a new account (Sign-up).

Registered User Requirements

These features are exclusive to authenticated users.

Personalization & Insights

1. The System must provide book recommendations based on the user's interaction history (favorite genres, authors) and global trends.
2. The System must generate a "Wrapped" style summary of the user's reading year (statistics, top genres).
3. The System must enable users to view a complete list of publications for a specific author.
4. The System must allow users to retrieve all reviews for a specific book on demand.

Content Creation & Library Management

5. The System must allow a user to review books, providing a numerical evaluation (score) and an optional written comment.
6. The System must allow each user to assign a status to a book, selecting from: *To-Read*, *Reading*, or *Read*.
7. The System must enable users to create and manage custom lists of books.

Social Interaction

8. The System must enable users to follow other profiles and view their latest activities in a dedicated feed.
9. The System must enable users to "like" or "unlike" specific books.
10. The System must enable users to "like" or "unlike" reviews written by others.
11. The System must allow users to view their list of friends/followed accounts.

Account Management

12. The System must permit users to Log In and Log Out securely.
13. The System must allow users to delete their account.

Administrator Requirements

Privileged operations for content and user management.

Catalog Management

1. The System must enable an Admin to add new books, authors, or genres to the database.
2. The System must enable an Admin to update existing information related to books, authors, or genres.
3. The System must enable an Admin to remove a specific book, author, or genre.

Moderation & Oversight

4. The System must allow an Admin to view the profile and data of any Registered User.
5. The System must enable an Admin to view any review posted on the platform.
6. The System must enable an Admin to delete reviews deemed inappropriate.
7. The System must enable an Admin to ban a Registered User¹.

¹Banning prohibits access but preserves specific user metadata to prevent re-registration with the same credentials.

2.2.3 Non-Functional Requirements

These requirements define the quality attributes of the system.

1. The System must follow RESTful design principles for API communication.
2. The System must implement backup and redundancy strategies to avoid permanent data loss.
3. The System must encrypt sensitive user data, specifically passwords, using industry-standard hashing algorithms.
4. The System must be highly available and fault-tolerant to ensure continuous service.
5. The System must enforce an Eventual Consistency model between the Document Database (MongoDB) and the Graph Database (Neo4j) to optimize performance over strict synchronization.

2.3 Expected Workload Distribution

A critical aspect considered when designing the system and database architecture is the expected workload profile. Given the social nature of the application, BookSphere is designed to handle a **Read-Heavy** workload, typical of large-scale social platforms.

2.3.1 Workload Profile (OLTP vs. OLAP)

The system operates primarily as an **OLTP (Online Transaction Processing)** environment, characterized by a high volume of short, atomic transactions (e.g., posting a review, following a user). However, features such as *Yearly Wrapped* and *Trending Probability* introduce periodic **OLAP (Online Analytical Processing)** workloads, requiring the aggregation of vast datasets.

The operational split is estimated as follows:

- **Read Operations (~80-85%):** The majority of traffic consists of content consumption: browsing book details, traversing the social graph (viewing friends' activities), searching the catalog, and viewing analytics dashboards. This read-heavy pattern is typical of social media platforms where content consumption vastly outpaces content creation.
 - *Architectural Implication:* This imbalance justifies the use of **Replica Sets** in MongoDB. By setting the read preference to `secondaryPreferred`, we offload the heavy read traffic to secondary nodes, keeping the Primary free for write operations. This configuration provides horizontal read scalability without compromising write consistency (detailed configuration is provided in Section 4.3.1).

- *Performance Optimization*: To further optimize read performance, frequently accessed data is denormalized (e.g., embedding popular reviews within book documents) and pre-aggregated statistics are computed asynchronously (e.g., `stats_per_year` buckets).
- **Write Operations (~15-20%)**: Writes are further categorized based on frequency and complexity:
 1. **High-Frequency Writes (~10-15%)**: Lightweight interactions such as 'Likes' and 'Follows'. These require low latency and can tolerate *Eventual Consistency*. Primary database: Neo4j for graph relationships.
 2. **Low-Frequency Writes (<5%)**: Content creation tasks such as writing detailed Reviews, rating books, and Administrative operations (CRUD on the catalog). These may require stricter validation and synchronous cross-database propagation to maintain data integrity.

Analytical Workload Considerations: While most operations are OLTP, certain features trigger OLAP-style queries:

- **Wrapped Generation**: Aggregates a user's entire reading history for the year, requiring full-scan of their review collection.
- **Revaluation of a Book**: Analyze the first and last year of review and see the ones with higher positive difference.
- **Dynamic Rankings**: Computes ordered lists of books based on calculated average ratings, filtered by genre, author, or year of the reviews.

To handle these mixed workloads, MongoDB's aggregation pipeline is leveraged for complex analytical queries, while indexes ensure that even full-collection scans can be executed efficiently. The system is designed to scale horizontally by adding more secondary replicas as read traffic grows.

2.4 UML Class Diagram

To provide a comprehensive view of the system's static structure and the relationships between entities, we present the UML Class Diagram in Figure 2.1.

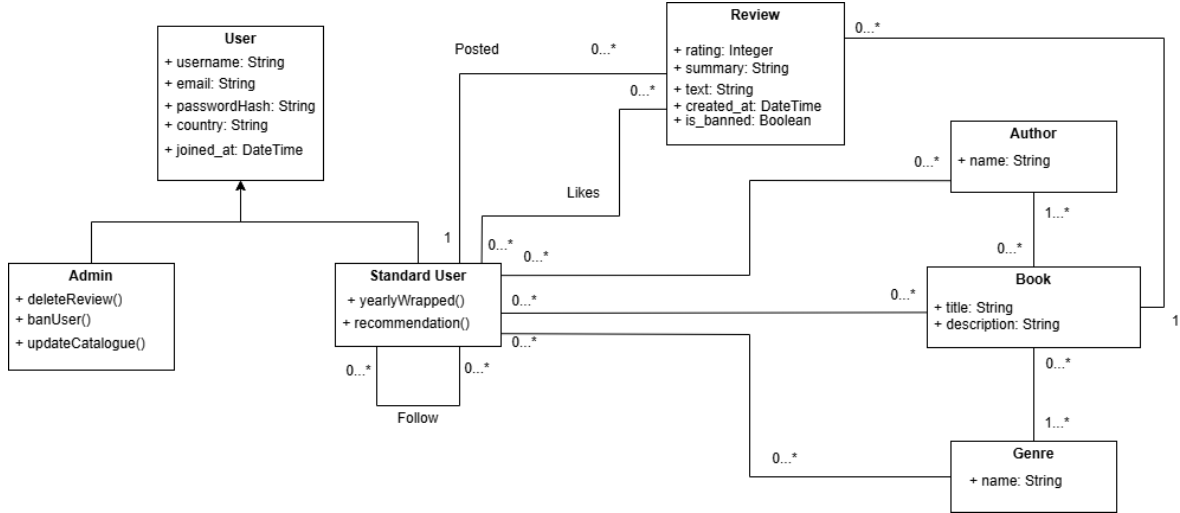


Figure 2.1: UML Class Diagram of BookSphere.

2.5 User Interface Concept and Mock-ups

This section presents the mock-ups developed to provide a conceptual visualization of the system’s user interface. These designs serve as a bridge between the functional requirements analyzed in the previous section and the final API implementation.

***Note:** These mock-ups are intended for informational purposes only and were not implemented in the final system due to the project’s focus on API-based architecture. However, they demonstrate how the defined requirements would translate into a frontend user experience.*

The following figures illustrate the interface design for the three main system actors: Unregistered Users, Registered Users, and Administrators.

2.5.1 Unregistered User Experience

The Guest interface focuses on **Search & Discovery** and **System Analytics**, allowing users to explore the catalog without barriers before converting via the **Access** requirements.

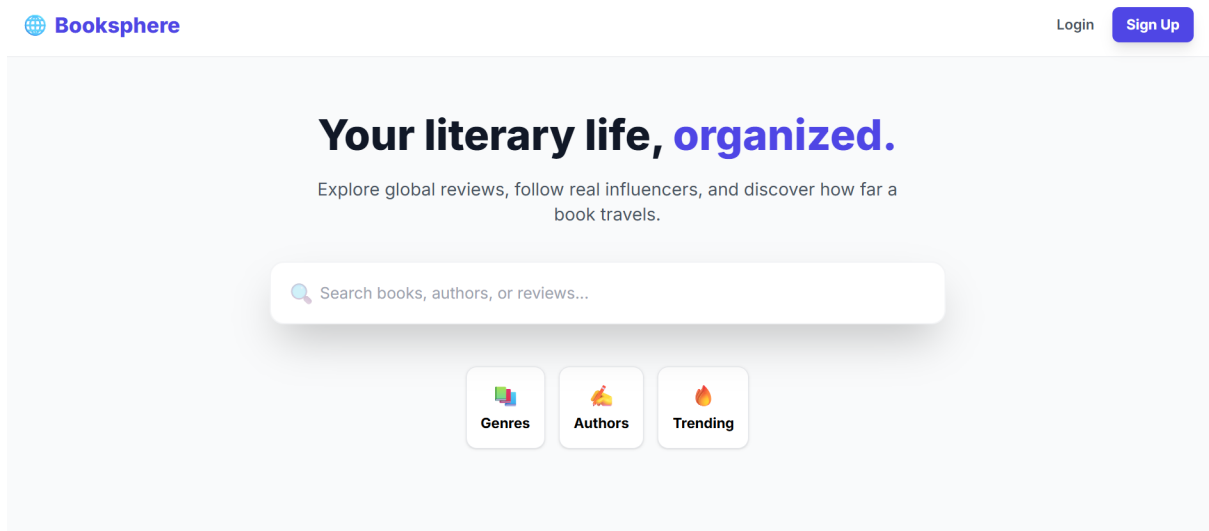


Figure 2.2: Search Engine
Satisfies Gen & Sys-Req 1-2-3

2.5.2 Registered User Experience

The Authenticated User interface enables Personalization, Content Creation, and Social Interaction. The designs below highlight the transition from passive reading to active engagement.

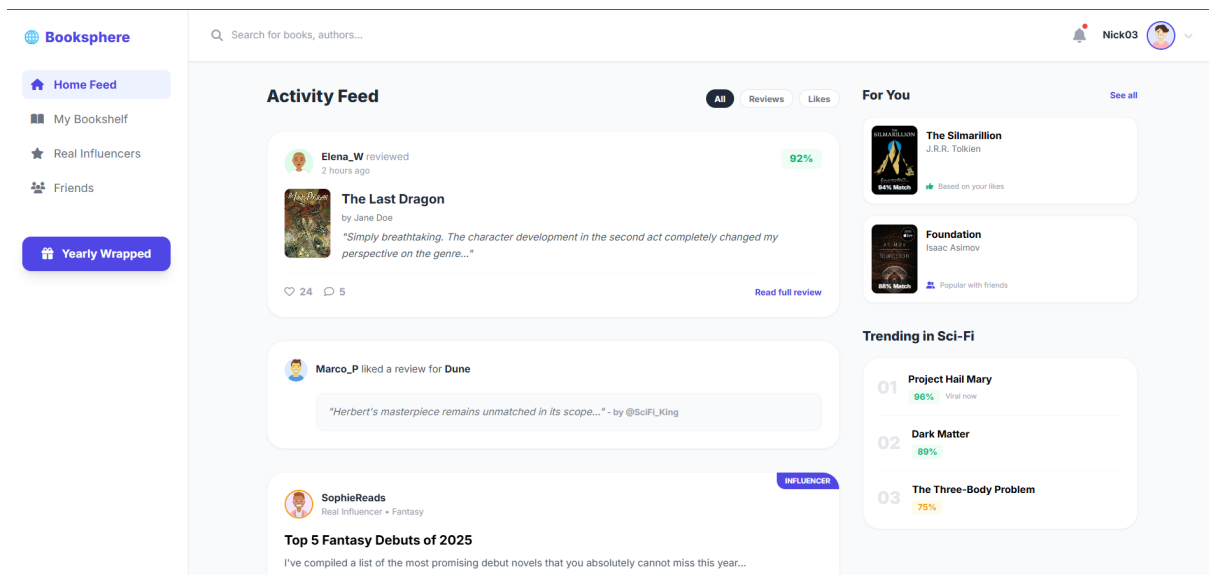


Figure 2.3: User's Feed
Satisfies Reg-Req 1 (Recs) & Req 8 (Follow Feed)

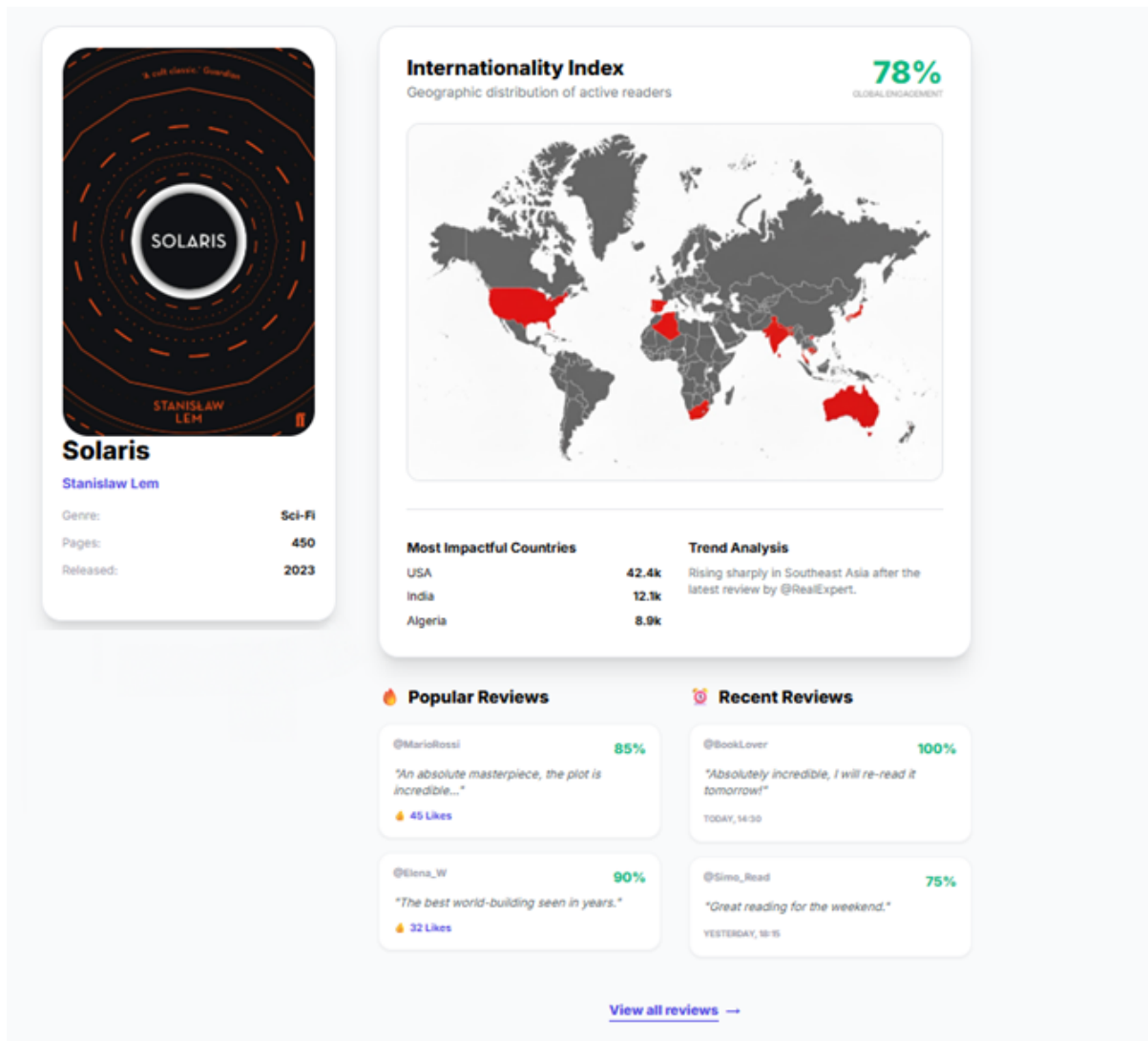


Figure 2.4: Review & Status Management
Satisfies Reg-Req 5 (Review/Score) & Req 6 (Status: To-Read/Read)



Figure 2.5: Personalized Analytics Dashboard.
Satisfies Reg-Req 2 (“Wrapped” Summary)

2.5.3 Administrator Experience

The Admin interface prioritizes Catalog Management and Moderation. These views are data-dense to facilitate efficient system oversight.

AdminSphere
Dashboard
Catalog

Catalog Management

Insert new content into the Booksphere database.

Add Book

TITLE

AUTHOR NAME

YEAR

SUMMARY

SAVE

Add Author

FULL NAME

REGISTER

Figure 2.6: Database Management
Satisfies Admin Req 1-3 (Add/Update/Remove)

AdminSphere
Dashboard
Catalog

User Management

View user profiles and manage access.

USER	ROLE	STATUS	ACTIONS
 @alice_r	Registered User	Active	View Profile Ban User
 @spammer_99	Registered User	Banned	View Profile

Review Moderation

Monitor and delete inappropriate reviews.

BOOK	USER	REVIEW	ACTIONS
The Last Dragon	@alice_r	This book was okay but the ending felt rushed...	View Delete
Space Odyssey	@spammer_99	CLICK HERE FOR FREE MONEY!!!!	View Delete

Figure 2.7: Moderation & Banning
Satisfies Admin Req 1-3 (Add/Update/Remove)

Chapter 3

Data Modeling

This chapter details the data persistence layer of **BookSphere**. Given the polyglot nature of the architecture, the data modeling strategy is split between a **Document-Oriented** approach (*MongoDB*) for content management and analytics, and a **Graph-Oriented** approach (*Neo4j*) for social interactions and recommendation algorithms.

3.1 Data Source and Volume

To ensure the system was tested against realistic data volumes and complex interaction patterns, we implemented a hybrid data generation strategy using custom automation scripts. The core catalog and foundational rating data are sourced from the **Amazon Books Reviews Dataset** (providing real-world book metadata and user ratings).

This real data is combined with a user base of approximately 10,000 active accounts. The generation process automatically populates the polyglot persistence layer:

- **Amazon Books Reviews Dataset** (~5GB) [1]: Provides the core catalog of books and real-world reviews spanning from 1996 to 2014.
- **Book-Crossing Dataset** [2]: Used as a supplementary source for user ratings to increase data *variety* and *volume*.

MongoDB (Document Store)	Neo4j (Graph Store - CSVs)
• 37.1MB for reviews • 27.7MB for books • 12.4MB for users • 3.5MB for authors	• 4.32MB for Nodes (User, Book, Review...) • 24.9MB for Relationships (LIKES, FOLLOWS...)

Table 3.1: Database Volumes by Collection.

3.2 Dataset Preprocessing

3.2.1 ETL Pipeline

The dataset generation follows a 6-step ETL (Extract, Transform, Load) pipeline implemented in two Python scripts:

1. **Books Merge:** Amazon `books_data.csv` and BookCrossing `Books.csv` are merged by title match. Configuration limits (`MAX_BOOKS=20,000`, `MAX_REVIEWS_PER_BOOK=50`, `MAX_USERS=12,000`) control the final dataset size.
2. **Users Generation:** Users are sampled from both sources, enriched with BCrypt-hashed passwords, randomized emails, and ISO country codes from 50 countries.
3. **Reviews Processing:** BookCrossing ratings (0–10 scale) and Amazon ratings (1–5 scale) are both normalized to the unified 0–100 scale used by BookSphere. Source provenance is preserved in the `source` field.
4. **Data Enrichment:** The pipeline injects computed fields: ISBNs are cross-matched, bookshelves are populated (`read + want_to_read`), and pre-aggregated statistics are calculated (`stats_per_year`, `month_score`, `popular_reviews_snapshot`, `author ratings_count/sum_ratings`).
5. **Text Normalization:** String cleaning operations—including trimming, special character removal, and format standardization—are applied to critical fields such as author names and book titles. This ensures data consistency and optimized search performance by resolving formatting discrepancies, guaranteeing that varying string representations refer to the same unique entity.
6. **MongoDB Export:** 4 JSONL files (`books.jsonl`, `users.jsonl`, `reviews.jsonl`, `authors.jsonl`) are generated for MongoDB ingestion via `mongoimport`.
7. **Neo4j Export:** A second script reads the MongoDB JSONL output and generates 14 CSV files (5 node types + 9 relationship types) for Neo4j’s `LOAD CSV` bulk import. Relationship inference is *smart*: genre likes are derived from the user’s actual reading history (top 3 genres), author likes from their bookshelf preferences, and book likes from high ratings (≥ 80).

3.2.2 Deployment Scripts

Three shell scripts automate database population:

- `populate_mongo.sh`: Single-node MongoDB import with index creation.
- `populate_local_cluster_mongo.sh`: 3-node replica set import (`localhost:27017`, `27018`, `27019`).
- `populate_neo4j.sh`: Copies CSVs to the import directory, clears the database, creates indexes on `mongoId/name`, and executes `LOAD CSV` for all nodes and relationships.

Data Variety: By merging Amazon and BookCrossing, the dataset satisfies the multi-source requirement. *Velocity:* Old reviews are excluded from trending and monthly analytics, satisfying the temporal relevance requirement.

3.3 Document Store: MongoDB

MongoDB acts as the primary "Source of Truth" for the system. It stores the complete information about books, users, and reviews. The schema design prioritizes **Read Performance** through specific NoSQL patterns.

3.3.1 Collections Overview

The database `booksphere.db` contains the following main collections:

1. **books:** The central catalog entity.
2. **users:** Stores authentication data and the user's library.
3. **reviews:** Stores the full text of reviews. Linked to books and users via references.
4. **authors:** Stores aggregated statistics about authors (e.g., total ratings).

3.3.2 Schema Design (JSON)

Books Collection

We implemented the **Bucket Pattern**¹ (`stats_per_year`) to make historical ranking queries instant, avoiding the need to scan millions of review documents. The **Snapshot Pattern**² (`recent_reviews_snapshot` and the `popular_reviews_snapshot`) allow the frontend to render the book details page with the latest reviews in a single query.

¹Instead of creating a separate document for every single rating statistic, this pattern groups (buckets) data by time intervals (e.g., years). In the text, it allows the system to retrieve historical ranking data instantly without having to scan and calculate over millions of individual review documents.

²It embeds the latest reviews so the frontend can render the book details page in a single query (avoiding a "join" with the reviews collection).

```

1 {
2   "_id": "698db76b4a1f5c8fa49331a8",
3   "title": "The Fellowship of the Ring",
4   "publication_year": 2000,
5   "description": "Begin your journey into Middle-earth...",
6   "availability": "ACTIVE",
7   "source": "amazon_master",
8
9   "author": {
10     "id": "698db76b4a1f5c8fa4933176",
11     "name": "J. R. R. Tolkien"
12   },
13
14   "genres": [
15     "Fiction"
16   ],
17
18   "external_ids": {
19     "isbns": [
20       "978-0547928210",
21       "... "
22     ]
23   },
24
25   "recent_reviews_snapshot": [
26     {
27       "_id": "698db7874a1f5c8fa493ddcf",
28       "user_id": "698db7854a1f5c8fa493cad6",
29       "username": "User_278550",
30       "rating": 98,
31       "summary": "",
32       "date": "2025-12-27T00:00:00.000+00:00"
33     }
34   ],
35
36   "stats_per_year": [
37     {
38       "year": 2011,
39       "ratings_count": 5,
40       "sum_rating": 90
41     },
42     {
43       "year": 2025,
44       "ratings_count": 1,
45       "sum_rating": 98
46     }
47   ],
48
49   "review_ids": [
50     "698db7874a1f5c8fa493ddcf",

```

```

51     "...",
52 ],
53
54 "month_score": {
55     "rating_count": 1,
56     "sum_rating": 98,
57     "Current_Month": "2025-12"
58 }
59 }

```

Listing 3.1: Example Book Document

Users Collection

The user document utilizes the **Subset Pattern**³ for the **bookshelf** to keep the document size manageable. Crucially, we maintain a **reviews_year** array (embedded) which is optimized specifically for the **Yearly Wrapped** aggregation, allowing the system to generate the report without joining the **reviews** collection.

```

1 {
2   "_id": "ObjectId('65d1...')",
3   "username": "User_12345",
4   "email": "user@bx.com",
5   "password_hashed": "...",
6   "status": "active",
7
8   "bookshelf": [
9     {
10      "book_id": "ObjectId('65b3f...')",
11      "status": "read",
12      "added_at": "ISODate('2023-06-01...')",
13      "title": "The Hobbit",
14      "author": { "name": "J.R.R. Tolkien" },
15      "genres": ["Fantasy"]
16    }
17  ],
18
19  "reviews_year": [
20    {
21      "id": "ObjectId('99a1...')",
22      "rating": 85,
23      "book": "Dune"
24    }
25  ]
26 }

```

Listing 3.2: User Document Structure

³Instead of storing every detail about every book a user has ever interacted with, the bookshelf stores a manageable subset of data (like title, author, status). This keeps the User document size small

Reviews Collection

The reviews collection stores the full textual content of user evaluations. It serves as the primary source for the "read more" functionality and provides the data for both the `recent_reviews_snapshot` in the books collection and the `reviews_year` array in the user documents.

```
1 {
2   "_id": "698db7874a1f5c8fa493ddcf",
3   "user_id": "698db7854a1f5c8fa493c8a5",
4   "username": "Rusneus003",
5   "rating": 83,
6   "source": "amazon",
7   "text": "This is, all-around, a pretty good lawn book...",
8   "summary": "Pretty good, although a few flaws",
9   "created_at": "2022-04-26T02:00:00.000Z",
10  "likes_count": 6,
11  "is_banned": false,
12  "book_snapshot": {
13    "book_id": "698db76e4a1f5c8fa493a4ec",
14    "title": "The Impatient Gardener's Lawn Book"
15  },
16  "author_snapshot": {
17    "id": "698db76e4a1f5c8fa493a4ed",
18    "name": "Jerry Baker"
19  }
20 }
```

Listing 3.3: Example Review Document

Authors Collection

The authors collection is designed to facilitate quick retrieval of author profiles and their complete bibliography. It utilizes the **Extended Reference Pattern** to list published books to help the visualization of an author book without scanning the book collection and the **Computed Pattern** to maintain real-time aggregate statistics.

```
1 {
2   "_id": { "$oid": "698db76c4a1f5c8fa4936c8d" },
3   "name": "Edgar Allan Poe",
4   "status": "ACTIVE",
5   "published_books": [
6     {
7       "_id": { "$oid": "698db76c4a1f5c8fa4936c8e" },
8       "title": "Tales of Mystery and Imagination"
9     },
10    {
11      "_id": { "$oid": "698db76e4a1f5c8fa4939a9c" },
12      "title": "The Narrative of Arthur Gordon Pym of Nantucket"
```



```

13     }
14 ],
15   "ratings_count": 14,
16   "sum_ratings": 1234
17 }

```

Listing 3.4: Example Author Document (Edgar Allan Poe)

3.3.3 Design Rationale

The MongoDB schema was engineered to minimize read latency for the most frequent query patterns, adhering to the principle of “store data together that is accessed together.” The adoption of specific NoSQL patterns addresses the following architectural requirements:

- **High-Performance Reads (Snapshot Pattern):** The book details page is the most visited section of the application. By embedding the `recent_reviews_snapshot` and `popular_reviews_snapshot` directly within the Book document, the system can render the complete page details—including metadata, author info, and highlighted reviews—with a single database fetch, eliminating the need for costly joins with the `reviews` collection.
- **Historical Analytics Optimization (Bucket Pattern):** Analyzing temporal trends (e.g., “how has this book’s rating changed over the last decade?”) typically requires scanning thousands of individual review documents. By pre-aggregating these metrics into the `stats_per_year` array bucket within the Book document, the system converts an $O(N)$ aggregation operation into an $O(1)$ document lookup.
- **Bounded Arrays (Subset Pattern):** While MongoDB allows arrays to grow, large unbounded arrays (like a user’s entire history of reviews) can degrade performance and exceed document size limits (16MB). We applied the Subset Pattern to the `User` collection by embedding only the current year’s activity (`reviews_year`). Older data remains accessible in the `reviews` collection, ensuring the User document remains lightweight and performant for authentication and session management.
- **Decoupling (Computed Pattern):** Author statistics (`ratings_count`, `sum_ratings`) are pre-computed. This decoupling ensures that displaying an author’s profile does not trigger a real-time aggregation across the entire catalog, shifting the computational load from read-time to write-time (during review insertion).

3.4 Graph Store: Neo4j

Neo4j is utilized strictly for modeling relationships and enabling high-performance traversal queries. The graph schema is designed to be lightweight: nodes primarily contain IDs linking back to MongoDB, while relationships carry timestamp data to support temporal queries.

3.4.1 Nodes and Labels

- **User:** Contains `mongoId`, `username`, `country`.
- **Book:** Contains `mongoId`, `title`, `year`.
- **Review:** Contains `mongoId`, `rating`, `createdAt`.
- **Author/Genre:** Lightweight nodes for structural categorization.

3.4.2 Relationships

Specific relationships are modeled to support the recommendation engine:

- `(:User)-[:FOLLOWS {since: ...}]->(:User)`: represents the social graph backbone.
- `(:User)-[:LIKES {timestamp: ...}]->(:Book)`: indicates a user likes a book.
- `(:User)-[:POSTED {timestamp: ...}]->(:Review)`: indicates a user posted a review.
- `(:User)-[:LIKES {timestamp: ...}]->(:Review)`: indicates a user likes another user's review.
- `(:User)-[:LIKES {timestamp: ...}]->(:Author)`: indicates a user likes an author.
- `(:User)-[:LIKES {timestamp: ...}]->(:Genre)`: indicates a user likes a specific genre.
- `(:Review)-[:REFER_TO]->(:Book)`: connects a review to the book being reviewed.
- `(:Book)-[:BELONGS_TO]->(:Genre)`: classifies a book into one or more genres.
- `(:Author)-[:WROTE]->(:Book)`: connects an author to their written book.

3.4.3 Graph Schema Visualization

The following diagram illustrates the complete graph schema with all node types and relationships:

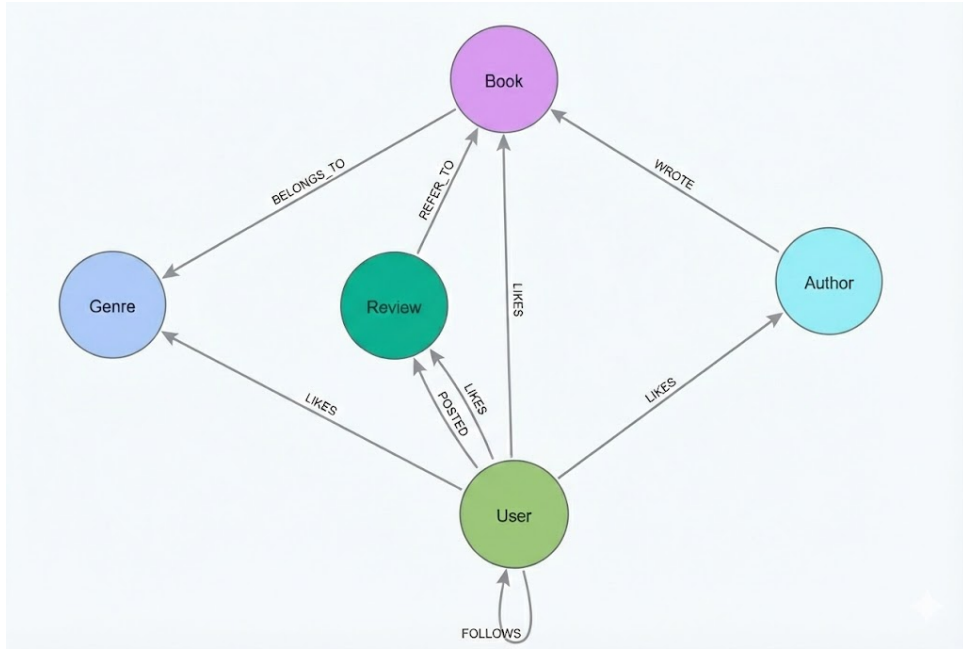


Figure 3.1: Neo4j Graph Schema showing all node types and relationships.

3.4.4 Design Rationale

The graph model was designed specifically to optimize the following query patterns:

- **Recommendation Queries:** Multi-hop traversals to find books liked by followed users or written by liked authors.
- **Social Network Queries:** Finding followers, followed users, and friend-of-friend relationships.
- **Analytics Queries:** Calculating internationality indices by aggregating country properties across relationship paths.
- **Influencer Detection:** Identifying users whose reviews receive high engagement through LIKES relationships.

The `mongoId` property on User, Book, Author, and Review nodes serves as the primary key for cross-database synchronization, enabling efficient lookups when coordinating operations between MongoDB and Neo4j.

Chapter 4

Databases Choices and Distributed Architecture

4.1 Persistence Strategy and Data Consistency

4.1.1 Polyglot Persistence Architecture

The application adopts a *Polyglot Persistence* approach to efficiently manage a highly connected, large-scale dataset. **MongoDB** was selected as the *Primary Document Database* to store the core entities (Users, Books, Authors, and Reviews) due to its high throughput in read/write operations and its flexible schema. Conversely, **Neo4j** was introduced as a *Secondary Graph Database* to seamlessly compute complex social interactions, recommendations, and graph-centric analytics (e.g., the Author Versatility Index).

Maintaining data integrity between a Document Store and a Graph Database requires a strictly defined synchronization strategy. Since distributed atomic transactions are not supported between MongoDB and Neo4j, *BookSphere* enforces consistency through application-level strategies, prioritizing **Eventual Consistency** for high-frequency social features while enforcing **Strict Consistency** (via compensation) for critical administrative operations.

4.1.2 Consistency Strategies

Strict Consistency (Synchronous)

For critical operations where data integrity is paramount (e.g., User Registration, Content Moderation), the system adopts a synchronous approach. The operation is considered successful only if it is correctly persisted in both databases.

- **Compensating Transactions (Manual Rollback):** If the write to the primary DB (MongoDB) succeeds but the propagation to Neo4j fails, a specific **try-catch** block intercepts the error and performs a manual rollback on MongoDB (deleting the just-created entity). This prevents "phantom data" (entities existing in Mongo but missing from the Graph).

- **Primary Failure:** If the write to the primary DB fails, the operation aborts immediately. No explicit `try-catch` block is needed for MongoDB writes, allowing the Spring `@Transactional` annotation to handle the rollback automatically.

Eventual Consistency (Asynchronous)

To minimize latency during write-intensive operations, the system adopts the **Eventual Consistency** paradigm specifically for heavy data denormalization tasks. Through the `@Async` annotation, these computationally expensive operations are decoupled from the main HTTP request thread:

- **Denormalization and Statistics:** The recalculation of aggregate metrics (e.g., `stats_per_year` buckets, Author total ratings) and the update of embedded snapshots (e.g., `recent_reviews_snapshot`) occur asynchronously. This strategy ensures that the user experience remains responsive (e.g., immediately acknowledging a posted review), while the system updates the read-optimized views in the background without blocking the response.

4.1.3 System Resilience

To ensure reliability in a distributed environment prone to temporary network latencies or lock contention, the system implements specific resilience patterns.

Spring Retry (`@Retryable`) has been enabled on some service methods. This allows the system to automatically retry failed operations (e.g., a temporary connection timeout with the Graph DB). Special care was taken to ensure the *idempotency* of these queries, guaranteeing that multiple re-executions do not generate duplicate records or inconsistent states.

4.1.4 Synchronization Matrix

To better illustrate the consistency guarantees across different domains, the operations are categorized into two matrices: **User-Initiated Operations** (characterized by high concurrency) and **Administrative Operations** (characterized by high data integrity requirements).

User and Social Domain

In this domain, the architecture often favors *Eventual Consistency* to ensure low latency and high availability for social interactions. However, core identity operations (like registration) maintain strict guarantees.

Table 4.1: Synchronization Matrix: User & Social Operations

Operation	Consistency	Strategy & Propagation	Primary DB
Registration	<i>Strict</i>	Synchronous write. Failure in Neo4j triggers a compensating transaction (delete) in MongoDB to ensure atomic user creation.	MongoDB
Post Review	<i>Hybrid</i>	Strict: Review text saved in Mongo and Node created in Neo4j synchronously. Eventual: Aggregation stats and embeddings occur via @Async.	MongoDB
Social Actions (Like/Follow)	<i>Eventual</i>	Write to Neo4j is blocking (immediate). Counter increments in MongoDB (e.g., likes_count) are propagated asynchronously.	Neo4j
Profile Update	<i>Eventual</i>	Update master record in Mongo. Propagation of username changes to graph nodes and review snapshots occurs in background.	MongoDB

Admin and Catalog Domain

In the administrative domain, data integrity is paramount. Operations are less frequent but require *Strict Consistency* to ensure the catalog remains the single source of truth across the distributed system.

Table 4.2: Synchronization Matrix: Admin & Catalog Operations

Operation	Consistency	Strategy & Propagation	Primary DB
Manage Book (Add/Edit)	<i>Strict</i>	Synchronous updates. Catalog data is propagated immediately to ensure search consistency.	MongoDB
Archive Book	<i>Strict</i>	Sets availability to ARCHIVED in Mongo and physically deletes the node in Neo4j to prevent recommendations for unavailable items.	MongoDB
Manage Author	<i>Eventual</i>	Updates the Author document in Mongo. Name changes propagate to the Graph node asynchronously.	MongoDB
Moderation (Ban User)	<i>Strict</i>	Synchronous execution. User status set to BANNED in Mongo; User node DETACH DELETE in Neo4j immediately to stop social graph access.	MongoDB

4.2 Indexes Justification and Validation

To guarantee optimal response times and avoid full collection scans (**COLLSCAN**), specific indexes have been designed and experimentally validated using the **explain("execution Stats")** method. The validation process compared the execution metrics (Total Keys Examined, Docs Examined, Execution Time) between indexed queries and unindexed scans.

4.2.1 MongoDB Indexes

Books Collection Indexes

Two primary indexes were applied to the books collection to support the analytics engine and the discovery features.

```
// 1. Genres Multikey Index - Supports filtering by genre in analytics
db.books.createIndex({ "genres": 1 });

// 2. Title Text Index - Supports full-text search
db.books.createIndex({ "title": "text" });
```

Listing 4.1: MongoDB Indexes for Books Collection

Validation 1: Analytics Aggregation (Genres)

The Multikey Index on the **genres** array proved crucial for the "Book Ranking" aggregation pipeline. As shown in Figure 4.1, the index allows the query engine to filter candidates without deserializing the entire catalog.



Figure 4.1: Impact of the Genre Index on Aggregation. The execution time drops significantly in (b) as the engine scans only relevant index keys.

Discarded Index: Availability Status

We analyzed the impact of indexing the **availability** field. Since over 95% of books are **ACTIVE**, the index has extremely low selectivity. The comparison below shows that using the index (b) results in a **totalKeysExamined** count almost identical to the number of documents, offering no performance benefit over a full scan (a) while introducing write overhead.

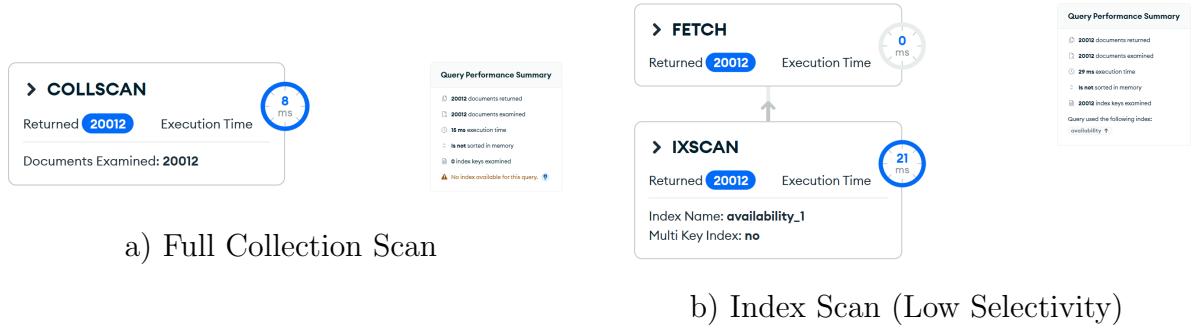


Figure 4.2: Discarded Index Analysis. The high cardinality of "ACTIVE" books renders the index ineffective, as the ratio of keys examined to returned documents approaches 1:1 in both cases.

Authors Collection Indexes

For search functionality, we compared a standard Regular Expression search against a MongoDB Text Index.

```
// 1. Author Name Text Index - Supports search and autocomplete
db.authors.createIndex({ "name": "text" });
```

Listing 4.2: MongoDB Indexes for Authors Collection

Validation 2: Text Search vs. Regex

The comparison highlights the performance gap. The Regex search (a) forces a COLLSCAN iterating all authors, whereas the Text Index (b) uses an inverted index structure for logarithmic time complexity lookups. *Note: Similar performance gains were observed and validated for the Book Title text index, which utilizes the same underlying mechanism.*

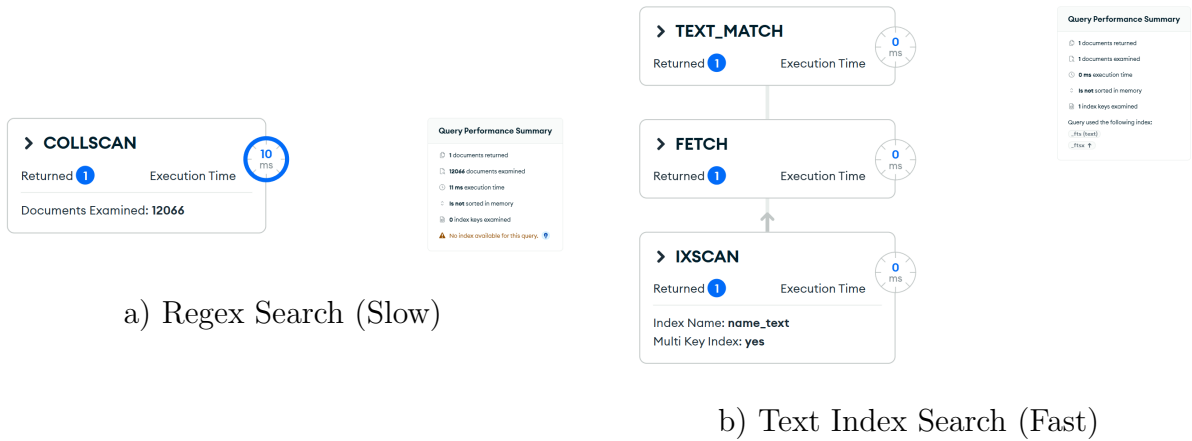


Figure 4.3: Performance comparison: Full Collection Scan using Regex vs. Text Search using Text Index.

4.2.2 Neo4j Indexes

In Neo4j, index usage was minimized to avoid unnecessary write overhead and maintain high insertion throughput. However, to support the Polyglot Synchronization strategy,

a *Range Index* was created exclusively on the `mongoId` property for all primary entities (User, Book, Review, Author) and on the `name` property for the Genre node.

Performance Impact on Synchronization

The `mongoId` index is critical for the "Bridge" operations between MongoDB and Neo4j. Without it, every synchronization event (e.g., a user liking a book) would require a full label scan to find the starting node, resulting in $O(N)$ complexity.

Representative Validation: User Lookup

The following comparison illustrates the execution plan for retrieving a specific User node by its `mongoId`. This operation is performed thousands of times during data ingestion. As shown in Figure 4.4, the unindexed query (a) forces a `NodeByLabelScan`, checking every single User node in the database. In contrast, the indexed query (b) utilizes a `NodeIndexSeek`, reducing the DB hits to a constant value ($O(1)$ or $O(\log N)$ depending on implementation), which is essential for real-time synchronization.

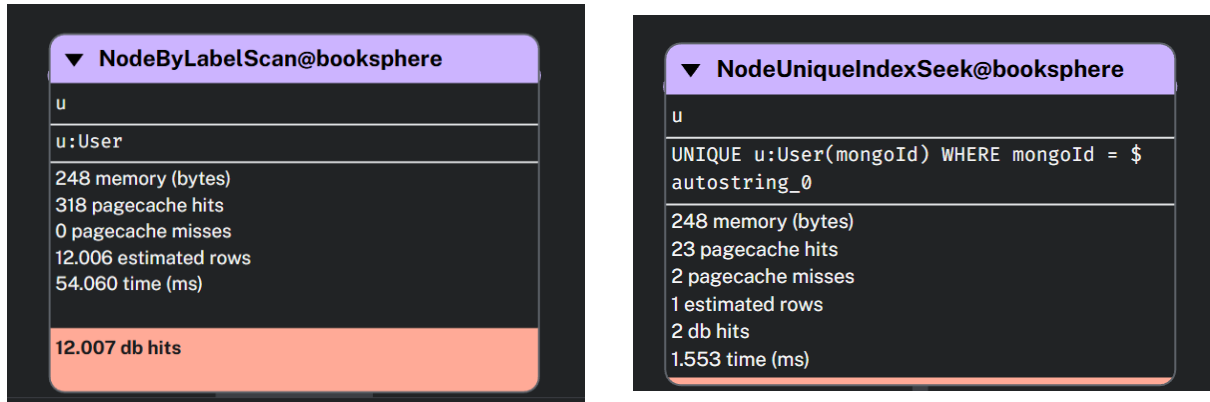


Figure 4.4: Query Plan comparison for `MATCH (u:User {mongoId: $id})`. The index transforms the operation from a linear scan of all users (millions of DB hits potential) to a direct index seek (single digit DB hits).

4.3 Virtual Machine Organization and Replica Sets

To guarantee high availability and fault tolerance, the database layer was deployed on the VIRTUAL LAB cluster provided by UNIPI.

4.3.1 Document Database Deployment

For the Document DB, a distributed architecture comprising a three-member replica set has been configured. This standard topology consists of one *Primary* node, which handles all write operations, and two *Secondary* nodes that asynchronously replicate the primary's oplog to ensure data redundancy.

The replica set initialization configuration (`rsconf`) assigns specific priorities to govern the election process. As shown below, the server at 10.1.1.77 is designated as the preferred primary (Priority 1) for the replica set named `lsmdb`:

```
rsconf = {
  _id: "lsmdb",
  members: [
    { _id: 0, host: "10.1.1.77:27018", priority: 1 },
    { _id: 1, host: "10.1.1.78:27019", priority: 2 },
    { _id: 2, host: "10.1.1.80:27020", priority: 5 }
  ]
};
rs.initiate(rsconf);
```

Listing 4.3: MongoDB Replica Set Configuration

4.3.2 Graph Database Deployment

Deploying a native Neo4j cluster typically requires an enterprise license. Therefore, a single instance of the graph database was deployed on the virtual cluster alongside the Document DB replicas. To optimize resource allocation and prevent interference with the primary MongoDB node's heavy write workloads, the Neo4j instance was co-located on one of the secondary servers.

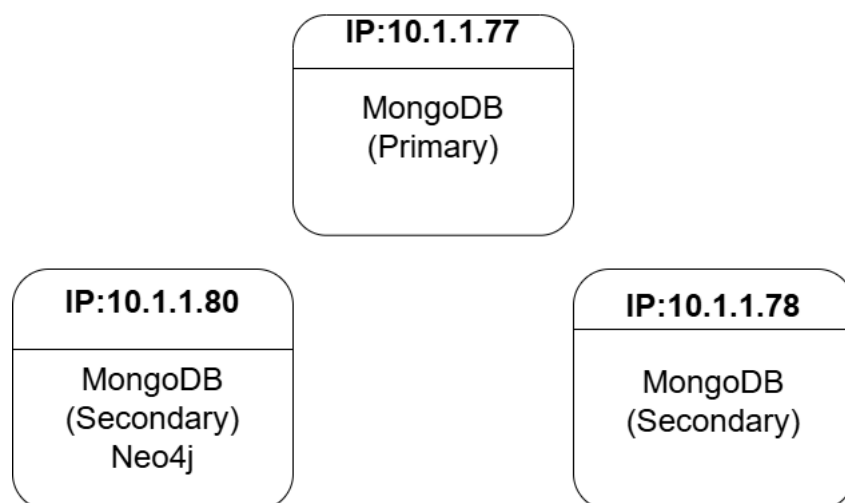


Figure 4.5: Overview of the Virtual Machines Organization and Database Deployment on the UNIPi Cluster.

4.4 Sharding Strategy

As the dataset grows beyond the capacity of a single replica set, *horizontal scaling* becomes necessary. Sharding in MongoDB divides collection data into chunks across servers to improve scalability, though it introduces overhead if queries span multiple shards. To implement this, a shard key must be selected.

The chosen shard keys for this system are `Reviews:_id` and `Users:username`. Both fields are mandatory and unique, ensuring a uniform distribution of documents across shards and enabling fair load balancing among nodes.

This design choice guarantees that write and read operations are evenly distributed, preventing hotspots and promoting horizontal scalability. Furthermore, the planned APIs are designed so that operations never require accessing data across multiple shards simultaneously, thereby avoiding cross-shard coordination overhead.

Sharding is applied specifically to the `Reviews` and `Users` collections because they are expected to grow significantly faster than the others. Distributing these high-growth collections across multiple shards improves performance and ensures the system can scale efficiently as data volume increases.

Regarding the partitioning algorithm, the **consistent hashing algorithm** is selected to ensure even distribution and supports nodes addition and removal. For reviews, hashing the `_id` prevents write hotspots if a specific book becomes viral. For users, hashing the `username` prevents data skew caused by common alphabetic distributions.

This configuration is largely theoretical, as the application is unlikely to require sharding immediately:

- **No Heavy Write Load:** The system handles a Read-Heavy workload (~80-85% reads), making sharding for write distribution unnecessary as Replica Sets can effectively spread read traffic.
- **Manageable Data Volume:** The core dataset is approximately 200MB, which fits comfortably within a standard replica set without needing horizontal partitioning.
- **Efficient Replica Set Performance:** A 3-node replica set configuration provides sufficient high availability and performance for the expected volume, avoiding the complexity of a sharded cluster.

4.5 CAP Theorem Considerations

Following the deployment on the distributed cluster, we analyzed the system's behavior regarding the CAP theorem (Consistency, Availability, Partition Tolerance).

By default, a MongoDB replica set strongly favors **CP** (Consistency and Partition Tolerance). If the primary node fails or becomes partitioned, the system refuses writes until a new primary is elected to prevent data divergence. However, to enhance *Availability* for read-heavy operations (e.g., fetching book catalogs or reading reviews), the application leverages a specific configuration:

- **Write Concern (w=majority):** Ensures durability and consistency. Writes are acknowledged only after propagating to a majority of nodes. This penalizes latency but favors data safety (CP behavior), allowing stale data (AP).
- **Read Preference (secondaryPreferred):** Shifts read operations towards an **AP** model. The application reads from secondary nodes when possible, accepting *Even-*

tual Consistency (reading slightly stale data) in exchange for higher throughput and availability, even if the primary is down.

The final connection string used in the Spring Boot application reflects this hybrid strategy and targets the **lsmdb** replica set:

```
spring.data.mongodb.uri=mongodb
://10.1.1.77:27018,10.1.1.78:27019,10.1.1.80:27020/booksphere?
replicaSet=lsmdb&readPreference=secondaryPreferred&retryWrites
=true&w=majority&wtimeoutMS=10000
```

Listing 4.4: Spring Boot MongoDB Connection URI

Chapter 5

Implementation

The complete source code for the project described in this chapter is available in the GitHub repository [3].

5.1 Technologies and Frameworks

The application is built using Java and the **Spring Boot** framework, chosen for its robust ecosystem and native support for building RESTful web services. To manage the interaction with the NoSQL databases, the project relies on **Spring Data MongoDB** and the **spring-boot-starter-data-neo4j** module, which seamlessly integrates the Neo4j Java driver. Furthermore, the application employs **MapStruct** for automated object mapping and **JSON Web Tokens (JWT)** to enforce stateless, role-based security.

Additionally, tools such as **Lombok** have been utilized to significantly reduce boilerplate code and facilitate secure, constructor-based dependency injection. Regarding security, the application leverages Spring Security's dedicated and integrated features to reinforce overall system protection. Finally, the project adopts a strict layered architecture, which is thoroughly described in Section 5.3.

5.2 Data Deletion Strategies and Lifecycle Management

In a distributed social platform like **BookSphere**, data deletion is a critical operation that requires balancing strict legal requirements (such as the "Right to be Forgotten") with the need to maintain referential integrity and statistical consistency. To address these conflicting goals, we adopted a hybrid strategy distinguishing between **Soft Delete** (logical removal) and **Hard Delete** (physical removal), depending on the entity type and the persistence layer involved. The following sections detail the specific implementation for the core entities.

5.2.1 User Unregistration: The Hybrid Anonymization Approach

When a user requests account deletion, the system must comply with privacy regulations (GDPR) without compromising the platform’s content history or the integrity of the social graph.

- **MongoDB (Soft Delete + Anonymization):** Physically deleting a user document would leave “orphan” references in the `reviews` collection. Instead, we implement a **Soft Delete** with strict anonymization. The user’s status is set to `DELETED`, and all PII (Personally Identifiable Information)—such as email, password hash, and full name—are overwritten with **empty strings** (`""`). This ensures data sanitization while avoiding null-pointer issues in the application layer. Crucially, the `username` field within their historical reviews is set to `null` (or a placeholder), effectively decoupling the content from the identity while preserving the text.
- **Neo4j (Graph Anonymization):** Unlike the standard deletion approach, in the Graph DB we choose to **preserve the User node** but clear its identity properties by setting the username to an **empty string**. This strategy maintains the graph structure/topology, allowing the recommendation algorithms (Collaborative Filtering) to leverage the user’s past interactions (likes/follows) without exposing their identity. These anonymous nodes are then filtered out from search results.

```
// MONGODB: Anonymize Document
user.setStatus(UserStatus.DELETED);
user.setEmail(""); // Clear PII with empty string
user.setUsername(""); // Clear identity
user.setPasswordHashed(""); // Clear security credentials

// Update Reviews (Decoupling)
reviewRepository.removeUsernameFromReviews(userId);

// NEO4J: Anonymize Node
// Keep the node for graph algorithms, but hide identity
userNode.setUsername("");
```

Listing 5.1: User Anonymization Logic

5.2.2 Review Deletion: Hard Delete with Cascade Consistency

Unlike users, reviews are content entities. When a review is removed (either by the author or by a moderator), it implies that the content is no longer valid or violates policies. Therefore, we adopt a **Hard Delete** strategy.

However, simply removing the document is insufficient due to the denormalized architecture. The deletion triggers a synchronous “Cascade Update” to restore consistency:

1. **Physical Removal:** The review document is deleted from MongoDB and the corresponding node is removed from Neo4j.

2. **Snapshot Update:** The review is removed from the `recent_reviews_snapshot` embedded in the parent Book document.
3. **Stats Recalculation:** The book's aggregated metrics (`stats_per_year`, `average_rating`) are recalculated to reflect the score removal.

5.2.3 Catalog Lifecycle (Book/Author): Archival Strategy

Books and Authors represent the core catalog. Even if an item is removed from sale or flagged as a duplicate, it typically remains part of users' historical reading logs.

- **Strategy:** We apply a **Soft Delete** in MongoDB by setting the availability status to `ARCHIVED`. Conversely, in Neo4j, we perform a **Hard Delete** of the node.
- **Reasoning:** The Soft Delete in MongoDB allows us to hide the item from search results while preserving integrity for users who have this book in their `bookshelf`. The Hard Delete in Neo4j ensures that the recommendation engine immediately stops suggesting unavailable items to other users.

5.3 Directory Structure and Modules

To ensure maintainability, scalability, and a clear separation of concerns, the codebase is organized following a strict Layered Architecture. The main packages and their responsibilities are detailed below.

5.3.1 Config

The `Config` module groups all the configuration classes required to bootstrap and secure the application environment. Key components include:

- **SecurityConfig:** Defines the role-based access rules and permissions for every exposed REST endpoint.
- **JwtAuthenticationFilter:** A custom filter that intercepts incoming HTTP requests to validate the signature and expiration (set to 24 hours) of the provided JWT.
- **JwtAuthenticationEntryPoint:** Manages authentication failures, gracefully handling unauthorized access attempts by returning standardized HTTP 401 (Unauthorized) responses.
- **DatabaseConfig:** Creates the transactional managers for the two databases.

5.3.2 Controllers

This package contains the REST controllers responsible for exposing the application's endpoints and handling incoming HTTP requests. To ensure a clear separation of concerns, these controllers delegate all the underlying business logic to the Service layer, thereby maintaining the presentation layer as lightweight and efficient as possible.

The API endpoints are logically divided into three main categories based on their access level: open (public APIs), registered (user-specific APIs, distinguished by the `/me/` prefix), and administrative (distinguished by the `/admin/` prefix).

Public Controllers

These controllers handle requests that do not require user authentication:

- **AuthController:** Manages user authentication, including login and logout functionalities.
- **AnalyticsController:** Implements the main system analytics and statistics, accessible without registration.
- **BookController:** Allows users to search for and retrieve book details by ID or title.
- **AuthorController:** Handles queries related to authors (e.g., retrieving author details or searching by name).
- **UserController:** Provides functionalities to search for registered users by their ID or username.
- **ReviewsController:** Fetches multiple reviews based on a provided list of identifiers.

Registered User Controllers (`/me/`)

These controllers expose functionalities reserved for authenticated users:

- **BookshelfController:** Enables users to manage their personal bookshelves (add, modify, or remove books).
- **ReviewController:** Allows authenticated users to create, update, or delete their personal reviews.
- **FollowController:** Manages the social graph, allowing users to follow/unfollow other users or retrieve the list of followed accounts.
- **LikeController:** Handles the interactions for liking or unliking specific content (e.g., reviews or books).
- **ProfileController:** Manages account settings, providing options to update the username or delete the account entirely.

- **UserFeaturesController:** Implements user-specific analytics and custom features, such as personalized book recommendations or yearly summaries (e.g., "Wrapped").

Admin Controllers (/admin/)

These controllers are restricted to users with administrative privileges:

- **AdminBookController & AdminCatalogController:** Used by administrators to manage the application's catalog, including adding or updating books, authors, and genres.
- **AdminModerationController:** Provides moderation tools to ban malicious users or remove inappropriate reviews.

Endpoint Architecture and Validation

As recommended by the project guidelines, we adopted the Spring framework to implement the RESTful APIs. Furthermore, annotations such as `@Operation` are used to seamlessly integrate with Swagger for complete API documentation.

The following snippet illustrates a typical controller method, demonstrating how requests are received, validated, and passed to the Service layer:

```
@Operation(
    summary = "Update a review",
    description = "Modify a previously posted review.
                  Updates both MongoDB and Neo4j."
)
@PatchMapping("/{reviewID}")
public ResponseEntity<ReviewDTO> updateReview(
    @Parameter(description = "MongoDB ObjectId of the
                           review", example = "99a1...")
    @PathVariable String reviewID,
    @Valid @RequestBody ReviewDTO reviewDTO
) {
    //Input Validation
    ValidationUtils.validateObjectId(reviewID, "reviewID");

    //Delegation to Service Layer
    ReviewDTO updated = reviewService.updateReview(reviewID,
        reviewDTO);
    return ResponseEntity.ok(updated);
}
```

Listing 5.2: Example of a Controller endpoint with input validation (Registered/ReviewController)

As demonstrated in the code above, user input validation is strictly enforced at the controller level prior to any business logic execution. To avoid redundancy in this documen-

tation, we omit further code examples for other endpoints. The architectural pattern remains highly consistent across the application: variations strictly depend on the required CRUD operation (mapped via standard annotations like `@GetMapping`, `@PostMapping`, `@PutMapping`, `@PatchMapping`, or `@DeleteMapping`) and the method of parameter extraction (using `@PathVariable`, `@RequestParam`, or `@RequestBody`).

5.3.3 Model

The `Model` package contains the domain entities that map directly to the database collections and graph nodes. The package is subdivided into two main categories: MongoDB entities and Neo4j entities. Each category follows specific design patterns to optimize data access and maintain consistency across the polyglot persistence architecture.

MongoDB Entities

MongoDB entities are annotated with `@Document` and use `@Field` annotations to map Java camelCase fields to the snake_case convention stored in MongoDB. The main entities include:

- **BookDocument**: Represents a book with comprehensive metadata, including embedded subdocuments for author information, external IDs, review snapshots, yearly statistics, and monthly trending scores.
- **RegisteredUser**: Models a registered user with authentication credentials, bookshelf items, and yearly review summaries.
- **Review**: Stores user reviews with rating, text, and denormalized book snapshots for efficient retrieval.
- **AuthorDocument**: Contains author information with aggregated statistics.

The following example demonstrates the `Review` entity, which implements the *Partial Embedding* strategy by having a minimal `BookSnapshot` to avoid expensive joins when displaying reviews in user feeds:

```
@Data
@NoArgsConstructor
@AllArgsConstructor
@Document(collection = "reviews")
public class Review {

    @Id
    private String id;

    @Field("user_id")
    private String userId;

    private String username;
    private Integer rating; // Unified scale 0-100
```

```

private String source; // "amazon", "bookcrossing"
private String text;
private String summary; // Only for Amazon reviews

@Field("created_at")
private Instant createdAt;

// PATTERN: Eventual Consistency (Updated asynchronously)
@Field("likes_count")
private Integer likesCount;

// PATTERN: Subset (Minimum book data to display review)
@Field("book_snapshot")
private BookSnapshot bookSnapshot;

// Nested class
@Data
@NoArgsConstructor
@AllArgsConstructor
public static class BookSnapshot {
    @Field("book_id")
    private String bookId;
    private String title;
}
}

```

Listing 5.3: Review Entity with Embedded BookSnapshot (Subset Pattern)

Neo4j Entities

Neo4j entities are annotated with `@Node` and use a `mongoId` property as the cross-database foreign key, enabling seamless integration between the document and graph databases. The main entities include:

- **UserNode**: Represents a user in the social graph with username and country properties.
- **BookNode**: Represents a book node with title and publication year.
- **AuthorNode**: Represents an author node with name.
- **GenreNode**: Represents a genre node with name.
- **ReviewNode**: Represents a review node with rating and timestamp.

A critical design decision was made to **not map relationships as Java entity fields**. Instead, relationships are created and queried exclusively through custom `@Query` Cypher statements in the Neo4j repositories. This approach prevents memory issues that would occur when loading super-nodes (e.g., popular genres connected to thousands of books) into the JVM heap.

The following example shows the `UserNode` entity structure:

```

@Data
@NoArgsConstructor
@AllArgsConstructor
@EqualsAndHashCode(onlyExplicitlyIncluded = true)
@Node("User")
public class UserNode {

    @Id
    @Property("mongoId")
    @EqualsAndHashCode.Include
    private String mongoId; // Cross-database foreign key

    @Property("username")
    private String username;

    @Property("country")
    private String country;
}

```

Listing 5.4: UserNode Entity with mongoId as Foreign Key

This lightweight structure ensures that Neo4j nodes remain minimal, storing only the essential properties needed for graph traversals, while the complete entity data resides in MongoDB.

5.3.4 DTO (Data Transfer Objects)

To decouple the internal domain models (Entities) from the data exposed through the APIs, the application makes extensive use of *Data Transfer Objects* (DTOs). This architectural pattern provides several key benefits:

- **Encapsulation:** Internal entity structure changes do not affect API contracts.
- **Security:** Sensitive fields (e.g., hashed passwords) are never exposed to clients.
- **Flexibility:** DTOs can aggregate data from multiple entities or databases.
- **Optimization:** DTOs can include computed fields (e.g., average ratings) without modifying entities.

The application defines 22 DTOs organized by functional domain, including:

- **Authentication DTOs:** LoginDTO, RegisterDTO, AuthResponseDTO
- **Entity DTOs:** BookDTO, AuthorDTO, UserDTO, ReviewDTO, GenreDTO
- **Summary DTOs:** BookSummaryDTO, ReviewSnapshotDTO, BookshelfItemDTO
- **Analytics DTOs:** RankingDTO, InfluencerDTO, InternationalityDTO, RecommendationDTO, WrappedDTO, BookTrendDTO

- **Statistics DTOs:** `StatsPerYearDTO`, `TrendScoreDTO`, `ReviewYearDTO`

For example, the `AuthResponseDTO` encapsulates the JWT token and user information returned after successful authentication:

```
public record AuthResponseDTO(
    String token,
    String userId,
    String username,
    String role
) {}
```

Listing 5.5: `AuthResponseDTO` for Authentication Response

5.3.5 Mappers

The `Mappers` directory contains `MapStruct` interfaces that automatically generate the boilerplate code required for bidirectional conversion between Entities and DTOs. `MapStruct` is a compile-time code generator that creates type-safe and performant mapping implementations, eliminating the need for manual, error-prone conversion code.

The application defines 7 mapper interfaces:

- **BookMapper:** Converts between `BookDocument` and `BookDTO`, including nested sub-documents
- **AuthorMapper:** Handles `AuthorDocument` $\leftrightarrow$ `AuthorDTO` conversions
- **UserMapper:** Maps `RegisteredUser` $\leftrightarrow$ `UserDTO`
- **ReviewMapper:** Converts `Review` $\leftrightarrow$ `ReviewDTO`
- **RankingMapper:** Maps projection results to `RankingDTO`
- **BookTrendMapper:** Converts trending data to `BookTrendDTO`
- **WrappedMapper:** Handles complex aggregation results for yearly summaries

The following example demonstrates the `BookMapper` interface, which includes custom mapping logic using `@AfterMapping` to calculate aggregated statistics:

```
@Mapper(componentModel = "spring")
public interface BookMapper {

    @Mapping(target = "isbn", source = "externalIds.isbns")
    @Mapping(target = "averageRating", ignore = true)
    @Mapping(target = "totalRatingsCount", ignore = true)
    @Mapping(target = "totalSumRating", ignore = true)
    @Mapping(target = "trendScore", source = "monthScore")
    BookDTO toDTO(BookDocument document);
```

```

/**
 * AfterMapping: Calculate overall statistics from yearly
 * stats
 */
@AfterMapping
default void calculateOverallStats(@MappingTarget BookDTO dto
    ,
                                BookDocument document) {
    if (document.getStatsPerYear() != null &&
        !document.getStatsPerYear().isEmpty()) {
        int totalRatingsCount = 0;
        int totalSumRating = 0;

        for (BookDocument.YearStat yearStat : document.
            getStatsPerYear()) {
            if (yearStat.getRatingsCount() != null) {
                totalRatingsCount += yearStat.getRatingsCount
                    ();
            }
            if (yearStat.getSumRating() != null) {
                totalSumRating += yearStat.getSumRating();
            }
        }

        dto.setTotalRatingsCount(totalRatingsCount);
        dto.setTotalSumRating(totalSumRating);

        // Calculate average rating
        if (totalRatingsCount > 0) {
            dto.setAverageRating(((double) totalSumRating /
                totalRatingsCount));
        }
    }
}

/**
 * Custom method for date conversion
 */
@Named("instantToString")
default String instantToString(Instant instant) {
    return instant != null ? instant.toString() : null;
}
}

```

Listing 5.6: BookMapper with Custom Post-Processing Logic

The `@AfterMapping` annotation allows developers to inject custom logic after the automatic field mapping is complete. This is particularly useful for computing derived fields (like average ratings) that require aggregation across multiple nested objects.

5.3.6 Repositories

This layer manages all data access operations and serves as the interface between the application's business logic and the underlying databases. The repository layer is organized into two main categories: MongoDB repositories and Neo4j repositories, each extending their respective Spring Data interfaces.

MongoDB Repositories

MongoDB repositories extend `MongoRepository` and provide standard CRUD operations plus custom query methods. The main repositories include:

- **BookRepository:** Manages book documents with custom queries for search and filtering
- **RegisteredUserRepository:** Handles user data with queries for authentication and profile management
- **ReviewRepository:** Manages review documents with filtering capabilities
- **AuthorRepository:** Handles author documents with search functionality

Additionally, the MongoDB repository package includes *Projection* interfaces used by Spring Data to capture and map intermediate results returned by complex aggregation queries:

- **RankingProjection:** Captures book ranking results with title, author, and average rating
- **BookTrendProjection:** Maps trending book data with monthly scores
- **WrappedAggregationResult:** Handles complex yearly summary aggregation results

Neo4j Repositories

Neo4j repositories extend `Neo4jRepository` and include custom `@Query` methods written in Cypher for complex graph traversals. This design decision was critical to avoid memory issues when dealing with super-nodes. The main repositories include:

- **UserNodeRepository:** Manages user nodes and FOLLOWS relationships
- **BookNodeRepository:** Handles book nodes and their relationships
- **AuthorNodeRepository:** Manages author nodes and WROTE relationships
- **GenreNodeRepository:** Handles genre nodes and BELONGS_TO relationships
- **ReviewNodeRepository:** Manages review nodes and REFER_TO, LIKES relationships

The Neo4j repository package also includes specialized *Projection* interfaces for graph query results:

- **RecommendationProjection**: Maps personalized book recommendations
- **InfluencerProjection**: Captures influencer analytics data
- **InternationalityProjection**: Maps geographic distribution data
- **UserFollowProjection**: Handles follow relationship data
- **BookLikeProjection**, **AuthorLikeProjection**, **GenreLikeProjection**, **ReviewLikeProjection**: Map like relationship data

The following example demonstrates the **UserNodeRepository** with custom Cypher queries for managing follow relationships and generating personalized recommendations:

```
@Repository
public interface UserNodeRepository extends Neo4jRepository<
    UserNode, String> {

    Optional<UserNode> findByMongoId(String mongoId);

    /**
     * Check if user A follows user B
     */
    @Query("MATCH (a:User {mongoId: $userId})-[r:FOLLOWS]->(b:
        User {mongoId: $userId}) " +
        "RETURN COUNT(r) > 0")
    boolean isFollowing(@Param("userId") String userId,
        @Param("userId") String userId);

    /**
     * Create FOLLOWS relationship between two users
     */
    @Query("""
        MATCH (a:User {mongoId: $followerId})
        MATCH (b:User {mongoId: $followedId})
        CREATE (a)-[r:FOLLOWS {since: $since}]->(b)
        RETURN r
        """)
    void createFollowsRelationship(
        @Param("followerId") String followerId,
        @Param("followedId") String followedId,
        @Param("since") LocalDateTime since
    );

    /**
     * Get all users followed by a user with pagination
     */
    @Query("""
```

```

MATCH (u:User {mongoId: $userId})-[r:FOLLOWS]->(followed:
    User)
RETURN followed.mongoId AS userId,
        followed.username AS username,
        followed.country AS country,
        r.since AS since
ORDER BY r.since DESC
SKIP $skip LIMIT $limit
""")
List<UserFollowProjection> getFollowedUsers(
    @Param("userId") String userId,
    @Param("skip") long skip,
    @Param("limit") int limit
);

/**
 * Generate personalized book recommendations
 * Based on: books liked by followed users, books by liked
 * authors,
 * books in liked genres
 */
@Query("""
MATCH (u:User {mongoId: $userId})
OPTIONAL MATCH (u)-[:FOLLOWS]->(:User)-[:LIKES]->(b1:Book
)
OPTIONAL MATCH (u)-[:LIKES]->(:Author)-[:WROTE]->(b2:Book
)
OPTIONAL MATCH (u)-[:LIKES]->(:Genre)<-[:BELONGS_TO]-(b3:
Book)
WITH collect(b1) + collect(b2) + collect(b3) AS
    recommendations, u
UNWIND recommendations AS book
WITH u, book
WHERE NOT EXISTS((u)-[:POSTED]->(:Review)-[:REFER_TO]->(
    book))
        AND book IS NOT NULL
WITH book, count(*) AS score
ORDER BY score DESC
LIMIT $limit
MATCH (a:Author)-[:WROTE]->(book)
RETURN book.mongoId AS bookId,
        book.title AS title,
        book.year AS publicationYear,
        score,
        collect(a.name) AS authors
""")
List<RecommendationProjection> getUserRecommendations(
    @Param("userId") String userId,
    @Param("limit") int limit
);

```

```

/**
 * Get or create UserNode - centralized pattern
 */
default UserNode getOrCreate(String mongoId, String username,
    String country) {
    return findByMongoId(mongoId)
        .orElseGet(() -> {
            UserNode newNode = new UserNode();
            newNode.setMongoId(mongoId);
            newNode.setUsername(username);
            newNode.setCountry(country);
            return save(newNode);
        });
}
}

```

Listing 5.7: UserNodeRepository with Custom Cypher Queries

This repository demonstrates several key patterns:

- **Custom Cypher Queries:** Complex graph traversals are expressed directly in Cypher using `@Query` annotations
- **Projections:** Results are mapped to lightweight projection interfaces instead of full entities
- **Default Methods:** The `getOrCreate` pattern is implemented as a default method to centralize common logic
- **Multi-hop Traversals:** The recommendation query traverses 2-3 hops in the graph to find relevant books

5.3.7 Services and Async

The `Services` package encapsulates the core business logic of the application. It orchestrates the flow of data, applies business rules, and coordinates interactions between repositories and external systems. The service layer is organized into 18 service classes, categorized by functional domain:

- **Public Services:** `AuthService`, `AnalyticsService`, `BookService`, `AuthorService`, `UserService`, `ReviewService`
- **User-Specific Services:** `BookshelfService`, `FollowService`, `LikeService`, `ProfileService`, `UserFeaturesService`
- **Admin Services:** `AdminBookService`, `AdminCatalogService`, `AdminModerationService`
- **Async Services:** `AsyncReviewTasks`, `AsyncLikeTasks`, `AsyncProfileTasks`, `AsyncAdminModerationTasks`

Public Domain Services

The **Public Services** layer handles the high-volume, read-heavy traffic generated by both guest and registered users. These services are optimized for throughput and reliability, serving as the primary interface for content discovery (Books, Authors, Analytics).

Below are two significant implementations that demonstrate the system's approach to complex data processing and fault tolerance.

Analytics Service: OLAP-style Aggregation The `AnalyticsService` is responsible for computing trends and insights. A prime example is the `getBookRevaluation` method, which identifies books that have significantly improved their reputation over time. Instead of fetching all reviews into memory (which would be inefficient), the service constructs a multi-stage **MongoDB Aggregation Pipeline** via `MongoTemplate`. This leverages the `stats_per_year` bucket pattern designed in the Data Modeling phase to perform the calculation entirely server-side.

```
/**
 * Get book revaluation trends
 * Performs complex MongoDB aggregation pipeline with multiple
 * stages
 */
@Retryable(
    retryFor = {RuntimeException.class},
    maxAttempts = 3,
    backoff = @Backoff(delay = 1000, multiplier = 2)
)
public List<BookTrendDTO> getBookRevaluation() {
    logger.info("Calculating book rating trends using dynamic
        averages");

    List<AggregationOperation> pipeline = new ArrayList<>();

    // 1. MATCH: Filter only ACTIVE books & with at least 2 years
    // of history
    pipeline.add(match(Criteria.where("availability").is("ACTIVE")
        .and("stats_per_year.1").exists(true))));

    // 2. PROJECT: Extract first (oldest) and last (newest)
    // snapshots
    pipeline.add(project()
        .andInclude("title")
        .and("author.name").as("author")
        .and(ArrayOperators.ArrayElemAt.arrayOf("
            stats_per_year").elementAt(0)).as("firstStat")
        .and(ArrayOperators.ArrayElemAt.arrayOf("
            stats_per_year").elementAt(-1)).as("lastStat")
    );

    // 3. PROJECT: Calculate start/end averages (handling
```

```

        division by zero)
pipeline.add(project()
    .and("title").as("title")
    .and("author").as("author")
    .and(ConditionalOperators.when(Criteria.where("
        firstStat.ratings_count").gt(0))
        .then(ArithmeticOperators.Divide.valueOf("
            firstStat.sum_rating")
            .divideBy("firstStat.ratings_count"))
        .otherwise(0.0)).as("startRating")
    .and(ConditionalOperators.when(Criteria.where("
        lastStat.ratings_count").gt(0))
        .then(ArithmeticOperators.Divide.valueOf("
            lastStat.sum_rating")
            .divideBy("lastStat.ratings_count"))
        .otherwise(0.0)).as("endRating")
);

// 4. ADD FIELDS: Compute rating delta
pipeline.add(AddFieldsOperation.addField("ratingDelta")
    .withValue(ArithmeticOperators.Subtract.valueOf("
        endRating").subtract("startRating"))
    .build());

// 5. SORT & LIMIT: Top 10 highest improvement
pipeline.add(sort(Sort.Direction.DESC, "ratingDelta"));
pipeline.add(limit(10));

Aggregation aggregation = newAggregation(pipeline);

// Execute aggregation on "books" collection
AggregationResults<BookTrendProjection> results =
    mongoTemplate.aggregate(
        aggregation, "books", BookTrendProjection.class
    );

return bookTrendMapper.toDtoList(results.getMappedResults());
}

```

Listing 5.8: Complex Aggregation Pipeline in AnalyticsService

Book Service: Resilience and State Handling The BookService illustrates the standard pattern for entity retrieval, emphasizing resilience and logical deletion handling. The `findById` method utilizes the `@Retryable` annotation to handle transient failures (e.g., temporary database connection drops) without propagating the error immediately to the client. Crucially, the configuration excludes business logic exceptions (`BookNotFoundException`, `BookArchivedException`) from the retry mechanism, as retrying these errors would be futile.

```

/**
 * Find book by ID

```

```

*
* @param id MongoDB ObjectId of the book
* @return BookDTO with book information including snapshot
*         reviews and statistics
* @throws BookNotFoundException if book is not found
*/
@Retryable(
    retryFor = {RuntimeException.class},
    noRetryFor = {BookNotFoundException.class,
        BookArchivedException.class},
    maxAttempts = 3,
    backoff = @Backoff(delay = 1000, multiplier = 2)
)
public BookDTO findById(String id) {
    logger.info("Finding book by id: {}", id);

    BookDocument book = bookRepository.findById(id)
        .orElseThrow(() -> {
            logger.warn("Book not found with id: {}", id);
            return new BookNotFoundException("Book not found
                with id: " + id);
        });

    if(book.getAvailability().equals("ARCHIVED")){
        logger.warn("Book ARCHIVED with id: {}", id);
        throw new BookArchivedException("Book is archived with id
            : " + id);
    }

    BookDTO bookDTO = bookMapper.toDTO(book);
    logger.info("Book found: {}", book.getTitle());
    return bookDTO;
}

```

Listing 5.9: Resilient Entity Retrieval in BookService

Asynchronous Services and Eventual Consistency

A dedicated **Async** sub-directory groups all asynchronous services, which are crucial for implementing the *Eventual Consistency* pattern. These services are annotated with **@Async** and execute in separate threads, allowing the main request to complete immediately while denormalized data is updated in the background.

The asynchronous approach provides several benefits:

- **Performance:** User-facing operations complete quickly without waiting for all database updates
- **Scalability:** Background tasks can be distributed across multiple threads or servers
- **Resilience:** Failures in background updates don't affect the primary operation

- **Consistency:** Denormalized data is eventually synchronized across collections

The following example demonstrates the `AsyncReviewTasks` service class specifically the `updateMonthScore` method, which handles eventual consistency updates when reviews are created, modified, or deleted on the field `month_score` on book document:

```
@Service
@RequiredArgsConstructor
public class AsyncReviewTasks {

    private static final Logger logger = LoggerFactory.getLogger(
        AsyncReviewTasks.class);
    private static final int MAX_RECENT_REVIEWS = 3;
    private final MongoTemplate mongoTemplate;

    /**
     * Other Method
     */

    /**
     * Update month score for trending analysis (ASYNC)
     * Handles month transitions and atomic increments
     */
    @Async
    public void updateMonthScore(Review review, Integer
        ratingDelta,
                                Integer countDelta) {
        String bookId = review.getBookSnapshot().getBookId();
        String currentMonth = YearMonth.now().toString();
        String reviewMonth = YearMonth.from(
            review.getCreatedAt().atZone(ZoneOffset.UTC)
        ).toString();

        if (!reviewMonth.equals(currentMonth)) {
            logger.info("Skipping month score update: review from
                {}, current {}",
                        reviewMonth, currentMonth);
            return;
        }

        Query query = new Query(Criteria.where("_id").is(bookId))
            ;
        BookDocument book = mongoTemplate.findOne(query,
            BookDocument.class);

        if (book != null) {
            Update update = new Update();
            BookDocument.MonthScore currentScore = book.
                getMonthScore();

            // Check for month transition
            boolean isNewMonth = currentScore == null ||
```

```

        currentScore.getCurrentMonth() ==
            null ||
        !currentScore.getCurrentMonth().
            equals(currentMonth);

    if (isNewMonth) {
        // New Month: Full reset
        BookDocument.MonthScore newScore = new
            BookDocument.MonthScore();
        newScore.setCurrentMonth(currentMonth);
        newScore.setRatingCount(Math.max(countDelta, 0));
        newScore.setSumRating(Math.max(ratingDelta, 0));
        update.set("month_score", newScore);
    } else {
        // Same Month: Atomic increment
        update.inc("month_score.rating_count", countDelta
        );
        update.inc("month_score.sum_rating", ratingDelta)
        ;
    }

    mongoTemplate.updateFirst(query, update, BookDocument
        .class);
    logger.info("Updated month score for book: {}",
        bookId);
}
}
}

```

Listing 5.10: AsyncReviewTasks Service for Eventual Consistency

This service demonstrates several critical patterns for eventual consistency:

- **@Async Annotation:** Methods execute asynchronously in a separate thread pool
- **Atomic Operations:** Uses MongoDB's \$inc, \$push, and \$addToSet operators for thread-safe updates
- **Array Filters:** The filterArray method enables targeted updates within embedded arrays
- **Slice Operator:** Maintains a fixed-size array (last 3 reviews) using slice(-3)
- **Temporal Logic:** Handles month transitions for trending calculations
- **Logging:** Comprehensive logging for debugging and monitoring async operations

The async pattern is consistently applied across the application whenever denormalized data needs to be synchronized, ensuring that the primary user operation completes quickly while maintaining data consistency in the background.

5.3.8 Utils

The `Utils` package provides shared helper classes and utilities utilized across different layers of the application. These utilities centralize common functionality and promote code reuse. The package includes four main components:

JwtUtil

The `JwtUtil` class is responsible for all JWT-related operations, including token generation, validation, and claim extraction. It uses the `jjwt` library (JSON Web Token for Java) to create and parse tokens with HMAC-SHA256 signatures.

Key features include:

- Token generation with custom claims (userId, role, status)
- Configurable expiration time (default: 24 hours)
- Secure signature validation using Base64-encoded secret keys
- Comprehensive error handling for expired, malformed, or invalid tokens

```
@Component
public class JwtUtil {

    @Value("${jwt.secret}")
    private String jwtSecret;

    @Value("${jwt.expiration:86400000}") // 24 hours
    private long jwtExpirationMs;

    private SecretKey getSigningKey() {
        byte[] keyBytes = Decoders.BASE64.decode(jwtSecret);
        return Keys.hmacShaKeyFor(keyBytes);
    }

    public String generateToken(String userId, String username,
                               String role, String status) {
        Map<String, Object> claims = new HashMap<>();
        claims.put("userId", userId);
        claims.put("role", role);
        claims.put("status", status);

        return Jwts.builder()
            .claims(claims)
            .subject(username)
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() +
                jwtExpirationMs))
            .signWith(getSigningKey())
            .compact();
    }
}
```

```

    }

    public String extractUserId(String token) {
        return extractClaim(token, claims -> claims.get("userId",
            String.class));
    }

    public Boolean validateToken(String token) {
        try {
            Jwts.parser()
                .verifyWith(getSigningKey())
                .build()
                .parseSignedClaims(token);
            return !isTokenExpired(token);
        } catch (SignatureException | MalformedJwtException |
            ExpiredJwtException | UnsupportedJwtException |
            IllegalArgumentException e) {
            logger.error("Token validation failed: {}", e.
                getMessage());
            return false;
        }
    }
}

```

Listing 5.11: JwtUtil Token Generation and Validation

SecurityUtils

The `SecurityUtils` class provides convenient static methods to retrieve information about the currently authenticated user from the Spring Security context. This eliminates the need to manually pass authentication objects through method signatures.

UserPrincipal

The `UserPrincipal` class represents the authenticated user entity within the Spring Security context. It implements `UserDetails` and encapsulates user information extracted from the JWT token.

Constants

The `Constants` class centralizes application-wide constants such as default values, configuration keys, and magic numbers, promoting maintainability and reducing code duplication.

5.3.9 Validation

This module centralizes request payload validation, ensuring data integrity before business logic execution. The validation layer includes both utility classes and custom val-

idators integrated with Spring's validation framework.

ValidationUtils

The `ValidationUtils` class provides static methods for common validation tasks, throwing `ValidationException` when validation fails:

```
public static void validateUsername(String username) {
    if (username == null || username.isBlank()) {
        throw new ValidationException("Username is required")
        ;
    }
    if (username.length() < 3 || username.length() > 50) {
        throw new ValidationException(
            "Username must be between 3 and 50 characters"
        );
    }
    if (!username.matches("[a-zA-Z0-9_-]+$")) {
        throw new ValidationException(
            "Username can only contain letters, numbers,
            underscores and hyphens"
        );
    }
}
```

Listing 5.12: Method of `ValidationUtils` for Username Validation

Custom Validators

The validation package also includes custom JSR-303 validators for domain-specific constraints:

- `@ValidObjectId`: Validates MongoDB `ObjectId` format (24-character hex string)
- `@ValidBookshelfStatus`: Ensures bookshelf status is one of the allowed values
- `@ValidCountryCode`: Validates ISO 3166-1 alpha-2 country codes
- `@ValidYear`: Ensures year values are within reasonable bounds

These custom validators integrate seamlessly with Spring's `@Valid` annotation, enabling declarative validation at the controller level.

5.3.10 Exceptions

To guarantee data integrity and provide meaningful feedback to clients, this module centralizes error handling through a global exception handler and a hierarchy of custom exceptions.

Custom Exception Classes

The application defines 14 custom exception classes, each representing a specific error condition:

- **Not Found Exceptions:** `UserNotFoundException`, `BookNotFoundException`, `AuthorNotFoundException`, `ReviewNotFoundException`, `GenreNotFoundException`
- **Conflict Exceptions:** `UserAlreadyExistsException`, `AlreadyExistsException`, `UserAlreadyBannedException`
- **Authorization Exceptions:** `InvalidCredentialsException`, `UnauthorizedOperationException`
- **Validation Exceptions:** `ValidationException`
- **State Exceptions:** `BookArchivedException`, `AuthorArchivedException`

GlobalExceptionHandler

The `GlobalExceptionHandler` class, annotated with `@RestControllerAdvice`, intercepts exceptions thrown by any controller and converts them into standardized HTTP responses. This centralized approach ensures that all exceptions are handled consistently, with appropriate HTTP status codes, structured error messages, and comprehensive logging for debugging and monitoring purposes.

5.4 MongoDB Relevant queries

To fulfill the analytical requirements of the application and handle the large dataset efficiently, several complex aggregation pipelines have been designed for the Document Database. These pipelines heavily leverage MongoDB's native operators to process and transform data directly within the database engine, minimizing data transfer and optimizing performance.

5.4.1 Ranking

This pipeline computes the ranking of books based on their average rating. It filters the statistics by a specific year, aggregates the total ratings, and enforces a minimum threshold to exclude statistical outliers. Finally, it calculates the exact average and sorts the results. Crucially, it leverages the **Bucket Pattern** structure (`stats_per_year`) defined in Section 3.3.2 to perform this calculation efficiently without accessing the `reviews` collection.

```
1 db.books.aggregate([
2   // 1. Initial Match
3   {
4     "$match": {
```

```

5         "availability": "ACTIVE"
6         // "author.id": {$id}, // (Optional)
7         // "genres": "Horror" // (Optional)
8     }
9 },
10 // 2. Filter Array (Year Logic)
11 {
12     "$project": {
13         "title": 1,
14         "author": 1,
15         "targetStat": {
16             "$filter": {
17                 "input": "$stats_per_year",
18                 "as": "stat",
19                 "cond": { "$eq": ["$$stat.year", 2023] }
20             }
21         }
22     }
23 },
24 // 3. Extract Totals (Flattening)
25 {
26     "$project": {
27         "title": 1,
28         "author": 1,
29         "totalRatings": { "$sum": "$targetStat.ratings_count"
30             },
31         "sumRating": { "$sum": "$targetStat.sum_rating" }
32     }
33 },
34 // 4. Threshold Filter
35 {
36     "$match": { "totalRatings": { "$gt": 5 } }
37 },
38 // 5. Calculate Average & Rename fields for DTO
39 {
40     "$project": {
41         "name": "$title",
42         "additionalInfo": "$author.name",
43         "totalRatings": 1,
44         "averageRating": { "$divide": ["$sumRating", "
45             $totalRatings"] }
46     }
47 },
48 // 6. Final Sort & Limit
49 { "$sort": { "averageRating": -1 } },
50 { "$limit": 25 }
51 ] )

```

Listing 5.13: Book Rankings Aggregation Pipeline

5.4.2 Revaluation

This query identifies the books with the most significant rating gap between their first and last year of reviews. It extracts the first and last statistical objects from the `stats_per_year` array, safely calculates their respective averages avoiding division by zero, and computes the delta. Like in the Ranking Query it exploits the Bucket Pattern.

```
1 db.books.aggregate([
2   // 1. Filter books with at least 2 years of history
3   { "$match": {
4     "stats_per_year.1": { "$exists": true },
5     "availability": "ACTIVE"
6   }},
7   // 2. Extract first and last stat objects
8   { "$project": {
9     "title": 1,
10    "author": 1,
11    "firstStat": { "$arrayElemAt": ["$stats_per_year", 0] },
12    "lastStat": { "$arrayElemAt": ["$stats_per_year", -1] }
13  }},
14  // 3. Calculate Averages (Safe Division: Sum / Count)
15  { "$project": {
16    "title": 1,
17    "author": "$author.name",
18    "startYear": "$firstStat.year",
19    "endYear": "$lastStat.year",
20    "startRating": {
21      "$cond": [
22        { "$gt": ["$firstStat.ratings_count", 0] },
23        { "$divide": ["$firstStat.sum_rating", "$firstStat.ratings_count"] },
24        0.0
25      ]
26    },
27    "endRating": {
28      "$cond": [
29        { "$gt": ["$lastStat.ratings_count", 0] },
30        { "$divide": ["$lastStat.sum_rating", "$lastStat.ratings_count"] },
31        0.0
32      ]
33    }
34  }},
35  // 4. Calculate the Delta (End - Start)
36  { "$addFields": {
37    "ratingDelta": { "$subtract": ["$endRating", "$startRating"] }
38  }},
39  // 5. Sort and Limit
40  { "$sort": { "ratingDelta": -1 } },
41  { "$limit": 10 }
```

42]);

Listing 5.14: Book Revaluation Aggregation Pipeline

5.4.3 Yearly Wrapped

This advanced pipeline generates an annual summary for a specific user. It leverages the `$facet` operator to perform multiple parallel analyses—such as extracting the most read authors, the favorite genres, and the best/worst rated books—in a single database roundtrip. The `$unwind` operator is essential here to accurately flatten and count the frequency of array elements (authors and genres) before grouping them. It exploits the Partial Embedding of the last year review and the bookshelf design in the Section 3.3.2.

```
1 db.users.aggregate([
2   // 1. MATCH: Filter by specific user
3   { "$match": { "_id": "CURRENT_USER_ID" } },
4
5   // 2. ADDFIELDS: Pre-calculate data (filter and sort internal
6   //   arrays)
7   { "$addFields": {
8     "best_book": {
9       "$arrayElemAt": [
10        { "$sortArray": {
11          "input": { "$ifNull": ["$reviews_year", []] }
12          "sortBy": { "rating": -1 } // DESC
13        } }, 0
14      ],
15      "worst_book": {
16        "$arrayElemAt": [
17          { "$sortArray": {
18            "input": { "$ifNull": ["$reviews_year", []] }
19            "sortBy": { "rating": 1 } // ASC
20          } }, 0
21        ],
22      },
23      "yearly_books": {
24        "$filter": {
25          "input": { "$ifNull": ["$bookshelf", []] },
26          "as": "b",
27          "cond": {
28            "$and": [
29              { "$eq": ["$$b.status", "read"] },
30              { "$gte": ["$$b.added_at", ISODate("2026-01-01T00:00:00Z")] },
31              { "$lte": ["$$b.added_at", ISODate("2026-12-31T23:59:59Z")] }
32            ]
29          }
30        }
31      }
32    }
33  ]
34  ]
35  ]
36  ]
37  ]
38  ]
39  ]
40  ]
41  ]
42 ] );
```

```

33         }
34     }
35 }
36 }},
37
38 // 3. FACET: Execute 3 parallel sub-pipelines
39 { "$facet": {
40     // Analysis 1: Top Authors
41     "top_authors": [
42         { "$project": { "yearly_books": 1 } },
43         { "$unwind": "$yearly_books" },
44         { "$group": {
45             "_id": "$yearly_books.author.name",
46             "count": { "$sum": 1 }
47         } },
48         { "$sort": { "count": -1 } },
49         { "$limit": 3 }
50     ],
51     // Analysis 2: Top Genres
52     "top_genres": [
53         { "$project": { "yearly_books": 1 } },
54         { "$unwind": "$yearly_books" },
55         { "$unwind": "$yearly_books.genres" },
56         { "$group": {
57             "_id": "$yearly_books.genres",
58             "count": { "$sum": 1 }
59         } },
60         { "$sort": { "count": -1 } },
61         { "$limit": 3 }
62     ],
63     // Analysis 3: Metadata (Best/Worst & Totals)
64     "meta": [
65         { "$project": {
66             "best_book": 1,
67             "worst_book": 1,
68             "total_books_read": { "$size": "$yearly_books" }
69         } }
70     ]
71 } }
72 ] )

```

Listing 5.15: Yearly Wrapped Aggregation Pipeline

5.5 Neo4j Relevant Queries

To leverage the highly connected nature of the dataset and satisfy the requirement for typical "on-graph" domain-specific queries, several advanced Cypher queries have been implemented. These queries navigate complex, multi-hop paths to calculate metrics and provide social features that would be computationally prohibitive in a standard Document

DB.

5.5.1 User Recommendations

This query provides personalized book suggestions by traversing a 2-to-3-hop path. The algorithm recommends books based on three main social factors: books liked by the users the current user follows, books written by authors the user likes, and books belonging to genres the user likes. To optimize the recommendation engine and prevent suggesting books the user has already read, a pre-query is executed on the MongoDB backend to extract the read books from the user's `bookshelf`. This list is then passed to Neo4j as a blacklist parameter (`$excludedMongoIds`).

```
MATCH (u:User {mongoId: $userId})
OPTIONAL MATCH (u)-[:FOLLOWS]->(:User)-[:LIKES]->(b1:Book)
OPTIONAL MATCH (u)-[:LIKES]->(:Author)-[:WROTE]->(b2:Book)
OPTIONAL MATCH (u)-[:LIKES]->(:Genre)-[:BELONGS_TO]->(b3:Book)
WITH collect(b1) + collect(b2) + collect(b3) AS recommendations, u
UNWIND recommendations AS book
WITH u, book
WHERE NOT EXISTS((u)-[:POSTED]->(:Review)-[:REFER_TO]->(book)) AND book
      IS NOT NULL
WITH book, count(*) AS score
ORDER BY score DESC
LIMIT $limit
MATCH (a:Author)-[:WROTE]->(book)
RETURN book.mongoId AS bookId,
       book.title AS title,
       book.year AS publicationYear,
       score,
       collect(a.name) AS authors
```

Listing 5.16: Cypher Query for User Recommendations

5.5.2 Internationality Index

This analytic calculates the "global reach" (Internationality Index) of a Book or an Author by analyzing the geographic origin (`country` property) of the users who interact with the entity. The traversal captures both users who posted a review and users who simply left a "Like".

```
// Internationality Index for a Book
MATCH (b:Book {mongoId: $bookId})
OPTIONAL MATCH (b)-[:REFER_TO]-(r:Review)-[:POSTED]-(reviewer:User)
OPTIONAL MATCH (b)-[:LIKES]-(liker:User)
WITH reviewer, liker
WITH collect(DISTINCT reviewer) + collect(DISTINCT liker) AS users
UNWIND users AS u
WITH u WHERE u IS NOT NULL AND u.country IS NOT NULL
RETURN u.country AS country,
       count(DISTINCT u.mongoId) AS uniqueUsers,
       count(*) AS totalInteractions
ORDER BY uniqueUsers DESC
```

```

// Internationality Index for an Author
MATCH (a:Author {mongoId: $authorId})
OPTIONAL MATCH (a)-[:LIKES]-(liker:User)
OPTIONAL MATCH (a)-[:WROTE]->(b:Book)-[:REFER_TO]-(r:Review)-[:POSTED]
    ]-(reviewer:User)
WITH liker, reviewer
WITH collect(DISTINCT liker) + collect(DISTINCT reviewer) AS users
UNWIND users AS u
WITH u WHERE u IS NOT NULL AND u.country IS NOT NULL
RETURN u.country AS country,
        count(DISTINCT u.mongoId) AS uniqueUsers,
        count(*) AS totalInteractions
ORDER BY uniqueUsers DESC

```

Listing 5.17: Cypher Query for Book and Author Internationality Index

5.5.3 Genre Influencer

This query aims to identify "True Influencers" within the platform, focusing on quality over quantity. It searches for users who write reviews that receive a high volume of "Likes" from other users, either globally or filtered by a specific genre. To prevent anomalies, the query enforces a minimum threshold of reviews and excludes soft-deleted or banned users (represented by an empty string username).

```

// Influencers by a specific genre
MATCH (g:Genre {name: $genreName})->[:BELONGS_TO]-(b:Book)-[:REFER_TO]
    ]-(r:Review)-[:POSTED]-(influencer:User)
MATCH (r)-[:LIKES]-(fan:User)
WITH influencer,
        count(DISTINCT r) AS numReviews,
        count(fan) AS totalLikes
WHERE numReviews > 3 AND influencer.username <> ""
RETURN influencer.username AS username,
        totalLikes AS totalEngagement,
        numReviews AS numReviews,
        toFloat(totalLikes) / numReviews AS avgLikesPerReview
ORDER BY avgLikesPerReview DESC
LIMIT $limit

// Global Influencers (All genres)
MATCH (r:Review)-[:POSTED]-(influencer:User)
MATCH (r)-[:LIKES]-(fan:User)
WITH influencer,
        count(DISTINCT r) AS numReviews,
        count(fan) AS totalLikes
WHERE numReviews > 5 AND influencer.username <> ""
RETURN influencer.username AS username,
        totalLikes AS totalEngagement,
        numReviews AS numReviews,
        toFloat(totalLikes) / numReviews AS avgLikesPerReview
ORDER BY avgLikesPerReview DESC
LIMIT $limit

```

Listing 5.18: Cypher Query for Genre Influencers

Chapter 6

System Testing and Performance Evaluation

6.1 Introduction

This chapter presents the multi-layered validation strategy for **BookSphere**, addressing the polyglot architecture through three complementary testing layers:

1. **Java Integration Testing:** JUnit 5 tests covering database interactions and business logic (**178 test methods**, 15 test classes).
2. **Performance Testing:** Gatling scenarios validating system behavior under load, stress, and spike conditions.
3. **End-to-End API Testing:** two Postman collections – **Storytelling** (63 API calls, 6 fictional users, BEFORE/AFTER analytics comparison) and **API Demonstration** (systematic endpoint coverage).

All tests execute against real database instances (MongoDB replica set and Neo4j cluster) without mocks, ensuring authentic end-to-end validation.

6.2 Java Integration Testing with JUnit 5

The backend test suite uses **JUnit 5** within a `@SpringBootTest` context, executing against real MongoDB and Neo4j instances without mocks. Tests validate end-to-end data flow from Service layer through Repositories to both databases.

6.2.1 Key Testing Principles

- **Real Database Testing:** Direct interaction with 3-node MongoDB replica set and Neo4j cluster (no Mockito).

- **Test Profile:** Custom `@TestProfile` activates `cluster` profile for database connectivity.
- **Deterministic Ordering:** `@TestMethodOrder` enforces `cleanup` → `setup` → `tests` sequence.
- **Namespace Isolation:** Unique `TEST_PREFIX` + `UUID` prevents test interference.
- **Security Simulation:** Manual `SecurityContextHolder` setup for authenticated endpoints.

6.2.2 Test Coverage by Access Tier

The test suite provides comprehensive coverage across all three access tiers of the application (Open, Registered, Admin), validating both functional correctness and cross-database consistency.

Table 6.1: Integration Test Coverage by Access Tier and Controller

Test Category	Test Class	Methods	Subtotal
Open Access Tier	AnalyticsControllerTest	19	63
	AuthControllerTest	9	
	AuthorControllerTest	9	
	BookControllerTest	9	
	ReviewControllerTest (open)	8	
	UserControllerTest	9	
Registered User Tier	BookshelfControllerTest	14	72
	FollowControllerTest	12	
	LikeControllerTest	16	
	ProfileControllerTest	7	
	ReviewControllerTest (registered)	15	
	UserFeaturesControllerTest	8	
Admin Tier	AdminBookControllerTest	14	43
	AdminCatalogControllerTest	14	
	AdminModerationControllerTest	15	
Total Test Methods			178
Total Test Classes			15

6.2.3 Cross-Database Consistency Validation

The tests explicitly verify synchronization between MongoDB and Neo4j for critical operations:

- **Review CRUD:** Validates embedded snapshots in MongoDB, Neo4j relationships (`POSTED`, `REFERS_TO`), and aggregation updates (`stats_per_year`, `month_score`).

- **Like Operations:** Confirms Neo4j LIKES relationships and MongoDB counter atomicity.
- **Ban Cascade:** Ensures user status change cascades to both databases (review cleanup, relationship deletion).

Test Execution: All 178 tests run in **3-5 minutes** on the Virtual Lab cluster using.

6.3 Performance Testing with Gatling

System behavior under realistic load conditions was validated using **Gatling 3.11.5**, a high-performance load testing framework. Tests execute against a fully populated database (~20k books, ~12k users, ~140k reviews) with MongoDB replica set (3 nodes) and Neo4j instance.

6.3.1 Test Results Summary

Table 6.2: Gatling Performance Test Results - Complete Metrics

Metric	BasicLoad	Stress	Spike	Endurance	DBIntensive	Total
Objective	Baseline perf.	Capacity limits	Traffic surge	Stability	Heavy queries	-
Load Pattern	30 VU	5→20 VU	10→30 spike	10 VU const	15 VU	-
Duration	90s	60s	90s	120s	120s	-
Requests	1260	390	2700	2987	600	7937
Mean RT	183ms	130ms	359ms	122ms	186ms	176ms
p_{75} RT	275ms	165ms	580ms	145ms	287ms	290ms
p_{95} RT	363ms	224ms	1050ms	195ms	405ms	447ms
p_{99} RT	490ms	310ms	2280ms	340ms	560ms	796ms
Max RT	868ms	430ms	3080ms	1128ms	1383ms	3080ms
Success Rate	100%	100%	100%	100%	100%	100%

6.3.2 BasicLoadTest - Baseline Performance

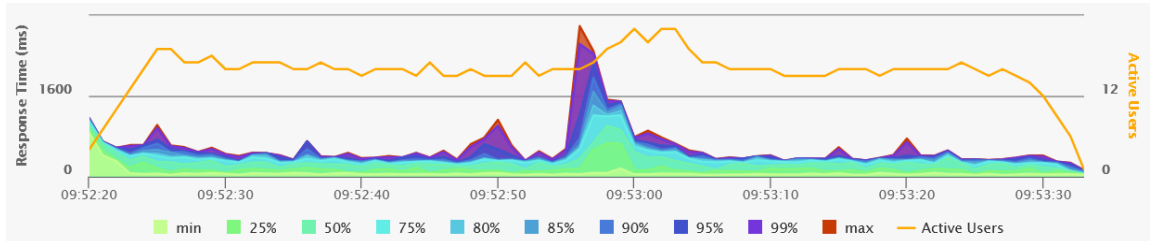


Figure 6.1: BasicLoadTest: Response Time Distribution and Active Users Over Time

6.3.3 StressTest - Capacity Validation

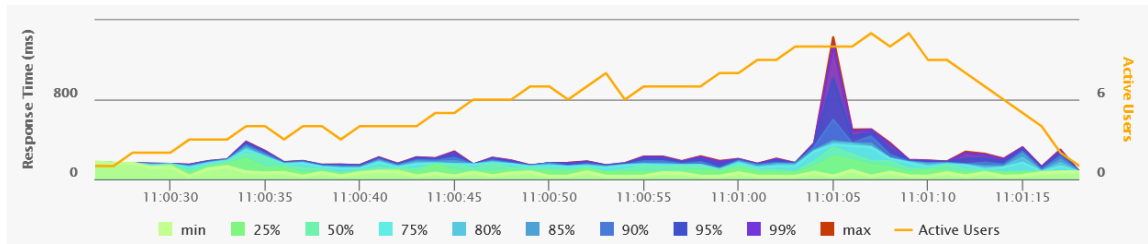


Figure 6.2: StressTest: Response Time Distribution with Progressive Load Increase

6.3.4 SpikeTest - Traffic Surge Resilience

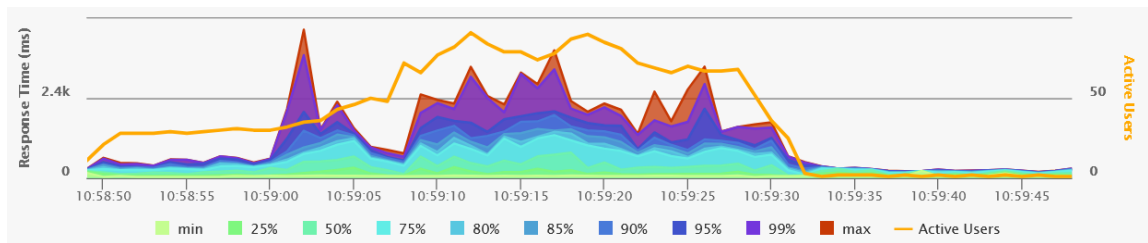


Figure 6.3: SpikeTest: Response Time Impact During Traffic Surge and Recovery

6.3.5 EnduranceTest - Long-Term Stability

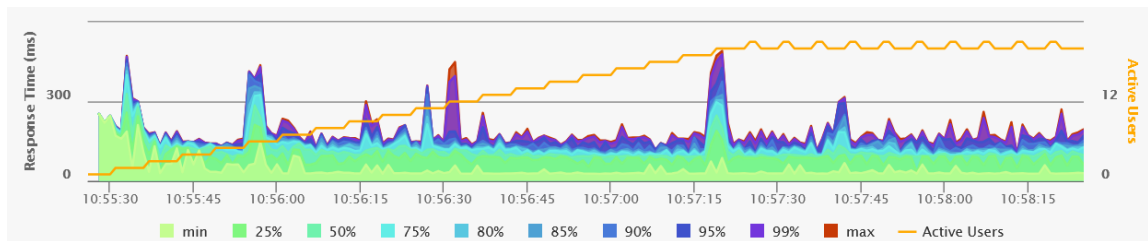


Figure 6.4: EnduranceTest: Stable Performance Under Sustained Load (No Degradation)

6.3.6 DatabaseIntensiveTest - Heavy Query Validation

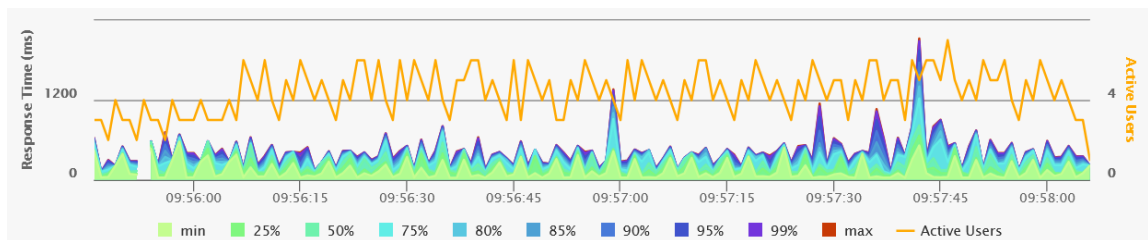


Figure 6.5: DatabaseIntensiveTest: Consistent Performance Under Complex Query Load

Test Execution Summary: All 5 scenarios executed successfully with 100% success rate and 7937 total requests. Complete interactive Gatling HTML reports with detailed percentile distributions are available in `Prperformance_test_clusterUNIPI/` directory.

6.4 End-to-End API Testing with Postman

Two Postman collections cover the end-to-end layer of the testing strategy. The first, described below, validates complete user journeys through a narrative scenario and measures the observable impact of user activity on the application analytics. The second provides a systematic demonstration of all individual API endpoints.

6.4.1 Storytelling with Analytics Comparison

Description

The collection “**BookSphere – Storytelling with Analytics Comparison BEFORE/AFTER**” simulates a realistic usage session through a theatrical narrative involving **6 fictional international users**. Rather than testing endpoints in isolation, the collection constructs a coherent story: users register, discover books, write reviews, assign likes, and form a social network – all in a single automated run of **63 API calls**.

The test is structured in 7 sequential phases:

Table 6.3: Phase Structure – Storytelling Collection

Phase Name		Actions	Req.
0	Prologue	Baseline capture: 4 public analytics (BEFORE)	4
1	Registrations	6 user registrations (US, GB, IT, FR, DE, ES)	6
2	Book Discovery	Retrieve 4 dynamic books from the catalog	4
3	Reviews	12 strategic reviews (3 per book × 4 books)	12
4	Likes	11 book likes + 6 author likes	17
5	Social Network	15 FOLLOWS relationships (dense network)	15
6	Epilogue	All 10 analytics AFTER + direct comparison	10
Tot.			68 (63 executed)

The Cast – 6 International Users:

- **Alice** (US) – Voracious novel reader
- **Bob** (GB) – Literary critic, thriller enthusiast
- **Charlie** (IT) – Bibliophile, philosophical readings
- **Dave** (FR) – Biography enthusiast

- **Eve** (DE) – Classics lover
- **Frank** (ES) – New author discoverer

Summary execution metrics:

Table 6.4: Newman – Global Execution Metrics

Metric	Value
Total requests	63
Failed requests	0 (100% success)
Assertions verified	42 (0 failed)
Total run duration	31.5s
Total data received	73.38 kB
Average response time	417ms

Database impact produced by the test: 12 **Review** documents inserted in MongoDB (**statsPerYear** and **monthScore** updated for 4 books); in Neo4j: 12 **ReviewNode** nodes, 11 **LIKES** on books, 6 **LIKES** on authors, 15 **FOLLOWS** relationships.

Analytics Changes: BEFORE vs. AFTER

Only the analytics that show difference after the population phase are reported below. Analytics whose results were identical before and after (Author Rankings, Revaluated Books, Internationality Author, Influencers) are omitted: the pre-existing dataset is too dense for 12 reviews and 11 likes to displace any entry in those rankings.

Table 6.5: Trending Books – BEFORE/AFTER

Pos.	BEFORE	AFTER
1	True Love (To Love Again)	True Love (To Love Again)
2	Life 101...	Life 101...
3	Night Woman	Night Woman
4	The Enchanted Forest Chronicles	The Enchanted Forest Chronicles
5	–	MongoDB Analytics Book 1
6	–	The Old Man and the Sea

Table 6.6: Book Rankings (year=2026) – BEFORE/AFTER Top 8

Pos.	BEFORE	AFTER
1	MDB Analytics Book 1 – Trending V4 (100.0)	MDB Analytics Book 1 – Trending V4 (100.0)
2	The Forgotten Scroll (100.0)	The Forgotten Scroll (100.0)
3	The Holy Grail (95.0)	The Holy Grail (95.0)
4	Camelot (92.0)	True Love (To Love Again) (92.74)
5	Camelot 2 (91.0)	Camelot (92.0)
6	–	Life 101... (90.67)
7	–	Night Woman (87.0)
8	–	The Enchanted Forest Chronicles (85.67)

Table 6.7: Rankings by Book IDs (3 test books) – BEFORE/AFTER

Pos.	BEFORE	AFTER
1	True Love – avg 92.74, 31 rev	True Love – avg 92.86, 35 rev
2	Night Woman – avg 87.0, 30 rev	Night Woman – avg 87.0, 33 rev
3	Life 101 – avg 84.56, 43 rev	Life 101 – avg 84.70, 46 rev

Table 6.8: Internationality – *True Love (To Love Again)*: countries BEFORE/AFTER

BEFORE	AFTER
US – 11 users, 22 interactions	US – 12 users, 24 interactions
DE – 10 users, 19 interactions	DE – 11 users, 21 interactions
IT – 10 users, 14 interactions	IT – 11 users, 16 interactions
GB – 10 users, 9 interactions	GB – 11 users, 11 interactions
FR – 5 users, 4 interactions	FR – 6 users, 6 interactions
ES – 3 users, 2 interactions	ES – 4 users, 4 interactions
CA – 1 user, 2 interactions	CA – 1 user, 2 interactions
6 countries, 50 users, 72 int.	7 countries, 56 users, 84 int.

Table 6.9: Personalized Recommendations for Alice – BEFORE/AFTER

BEFORE	AFTER
<i>(none – user did not exist)</i>	Night Woman: Night Woman (score 3) The Enchanted Forest Chronicles (score 1)

Table 6.10: Yearly Wrapped (Alice) – BEFORE/AFTER

BEFORE	AFTER
<i>(user did not exist)</i>	Best book: True Love (To Love Again) – 95/100 Worst book: Life 101... – 89/100 Total books read: 0 Top authors / genres: <i>(empty)</i>

6.4.2 Complementary Live Demo Collection

Description

The collection “**BookSphere – Live Demo (Complementary)**” is designed to be executed *after* the Storytelling collection, so that real entities (books, authors, users, reviews) already exist in the database. Its goal is twofold: cover every REST endpoint *not* exercised by the storytelling run, and systematically validate error-handling behavior through a dedicated suite of negative test cases.

The collection is organised in 8 sequential sections for a total of **76 requests**:

Table 6.11: Section structure – Complementary Live Demo Collection

Sec.	Name	Goal	Req.
A	Setup & Login	Register demo users, login as admin/user, harvest live <code>book_id</code> and <code>author_id</code> for downstream requests	6
B	Browse / Catalogue	Exercise all open-access read endpoints: book/author/user lookups, text-search, bulk review fetch, analytics filtered by year / genre / author	13
C	User Profile & Social	Full lifecycle: profile update, bookshelf CRUD (add → status changes → remove), follow/unfollow, personalized recommendations, Yearly Wrapped	10
D	Reviews	Registered CRUD cycle: create, read, update, like/unlike review, delete	7
E	Likes	Full like/unlike cycle for all three entity types: book, author, genre	9
F	Admin Panel	Admin operations: create/update author, create genre, create/update/delete book, list users and reviews, ban a user	10
G	Error Scenarios	20 negative cases covering HTTP 400, 401, 403 and 404 across all endpoint categories	20
Z	Cleanup	Self-deletion of the demo account	1
Tot.			76

Error Scenario Coverage (Section G)

Section G is the centerpiece of the complementary collection. It verifies that the API responds with the correct HTTP status code and a meaningful error message for every class of invalid input or unauthorized access. The 20 cases are grouped by error type:

- **400 – Bad Request / Validation:** duplicate username; missing required registration fields; invalid country code; weak password (no uppercase); review rating

out of range; missing `bookId`; invalid bookshelf status value; mutually exclusive analytics filters (`authorId` + `genre` simultaneously); malformed MongoDB `ObjectId` as path parameter; like with invalid `ObjectId`; publication year in the future; empty review ID list.

- **401 – Unauthorized:** wrong login credentials; access to a protected endpoint without a JWT token.
- **401/403 – Insufficient Privileges:** access to an admin endpoint with a regular user token; attempt to login by a banned user.
- **404 – Not Found:** non-existent book, author, and user lookups; ban request for a non-existent user ID.

Chapter 7

API Documentation

The BookSphere backend exposes a comprehensive RESTful API that adheres to industry-standard design principles. All endpoints are documented using **OpenAPI 3.0** (Swagger) specifications and are accessible through an interactive UI at `/swagger-ui.html` when the application is running.

7.1 API Architecture

The API is organized into three main categories based on authentication requirements:

1. **Open Endpoints** (`/api/v1/...`): Public access, no authentication required
2. **Registered User Endpoints** (`/api/v1/me/...`): Require JWT authentication
3. **Admin Endpoints** (`/api/v1/admin/...`): Require admin role JWT

For comprehensive implementation details, including code organization, package structure, and design patterns, refer to Chapter 5. The complete API specification with request/response schemas is accessible via the interactive Swagger UI at `http://localhost:8080/swagger-ui.html` when the application is running

7.2 Authentication Mechanism

The API uses **JWT (JSON Web Tokens)** for stateless authentication:

```
// 1. Login Request
POST /api/v1/auth/login
Content-Type: application/json

{
  "username": "alice",
  "password": "securePassword123"
}
```

```
// 2. Server Response (JWT issued)
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "userId": "65d1a2b3c4e5f6789012345",
  "username": "alice",
  "role": "USER"
}

// 3. Authenticated Request (token in Authorization header)
GET /api/v1/me/bookshelf
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Listing 7.1: JWT Authentication Flow

The JWT-based authentication mechanism is implemented using Spring Security and follows industry best practices. By utilizing JWTs, the system achieves a stateless authentication architecture, eliminating the need for server-side session storage. Tokens are signed using the HMAC-SHA256 algorithm and include claims for user identification, role authorization, and expiration time. The implementation details are provided in Chapter 5, Section 5.3.8

7.3 Key Endpoints Overview

7.3.1 Open Endpoints

Method	Endpoint	Description
POST	/api/v1/auth/register	Create new user account
POST	/api/v1/auth/login	Authenticate and receive JWT
GET	/api/v1/books/{id}	Get book details with snapshots
GET	/api/v1/books?title=&page=&size=	Search books by title (paginated)
GET	/api/v1/authors/{id}	Get author with bibliography
GET	/api/v1/authors?author_name=&page=&size=	Search authors by name
GET	/api/v1/users/username/{username}	Get user by username
GET	/api/v1/users/id/{id}	Get user by ID
GET	/api/v1/reviews?review=id1&review=id2	Get reviews by IDs (max 100)
GET	/api/v1/analytics/rankings/trendingbooks	Trending books (month score)
GET	/api/v1/analytics/books?year=&author=&genre=	Book rankings (filtered)
GET	/api/v1/analytics/rankings/authors	Author rankings (all-time)
GET	/api/v1/analytics/rankings/byBookIds?book=id1&book=id2	Book rankings given a list of IDs
GET	/api/v1/analytics/books/revaluated	Books with biggest rating deltas
GET	/api/v1/analytics/internationality/{id}?entityType=	Global reach of book/author
GET	/api/v1/analytics/influencers?genre=&limit=	Top influencers (Neo4j)

Table 7.1: Open API Endpoints (No Authentication Required)

7.3.2 Registered User Endpoints

Method	Endpoint	Description
POST	/api/v1/me/reviews	Create a review
PATCH	/api/v1/me/reviews/{id}	Update existing review
DELETE	/api/v1/me/reviews/{id}	Delete own review
POST	/api/v1/me/bookshelf	Add book to bookshelf
PATCH	/api/v1/me/bookshelf/{bookID}	Update book status
DELETE	/api/v1/me/bookshelf/{bookID}	Remove from bookshelf
POST	/api/v1/me/follow	Follow a user
DELETE	/api/v1/me/unfollow/{userId}	Unfollow a user
GET	/api/v1/me/friends?page=&size=	Get followed users
POST	/api/v1/me/like/book	Like a book
DELETE	/api/v1/me/unlike/book/{bookID}	Unlike a book
POST	/api/v1/me/like/review	Like a review
DELETE	/api/v1/me/unlikes/review/{id}	Unlike a review
POST	/api/v1/me/likes/genres	Like a genre
DELETE	/api/v1/me/unlike/genres/{name}	Unlike a genre
POST	/api/v1/me/likes/authors	Like an author
DELETE	/api/v1/me/unlike/authors/{id}	Unlike an author
GET	/api/v1/me/liked/book?page=&size=	Get liked books
GET	/api/v1/me/liked/author?page=&size=	Get liked authors
GET	/api/v1/me/liked/review?page=&size=	Get liked reviews
GET	/api/v1/me/liked/genre?page=&size=	Get liked genres
PATCH	/api/v1/me/username	Update username
DELETE	/api/v1/me/account	Delete own account
GET	/api/v1/me/recommendations?limit=	Personalized recommendations
GET	/api/v1/me/wrapped	Yearly personal recap

Table 7.2: Registered User API Endpoints (JWT Required)

7.3.3 Admin Endpoints

Method	Endpoint	Description
POST	/api/v1/admin/books	Add new book to catalog
PUT	/api/v1/admin/books/{id}	Update book information
DELETE	/api/v1/admin/books/{id}	Soft-delete book (ARCHIVED)
POST	/api/v1/admin/authors	Add new author
PUT	/api/v1/admin/authors/{id}	Update author information
DELETE	/api/v1/admin/authors/{id}	Soft-delete author (ARCHIVED)
POST	/api/v1/admin/genres	Add new genre (Neo4j only)
DELETE	/api/v1/admin/reviews/{id}	Delete any review (moderation)
PATCH	/api/v1/admin/users/{id}/ban	Ban a user (cascade)
GET	/api/v1/admin/users?page=&size=	List all users (paginated)
GET	/api/v1/admin/reviews?page=&size=	List all reviews (paginated)

Table 7.3: Admin API Endpoints (Admin JWT Required)

In total, the BookSphere backend exposes **52 API endpoints** across the three access tiers, covering complete CRUD operations for all entities, social interactions, analytics, and administrative moderation.

7.4 API Examples

7.4.1 Example 1: User Registration and First Review

```
# Step 1: Register new account
POST http://localhost:8080/api/v1/auth/register
Content-Type: application/json

{
  "username": "bookworm42",
  "email": "bookworm@example.com",
  "password": "SecurePass123!",
  "name": "Jane",
  "surname": "Doe",
```



```

    "country": "Italy"
}

# Response: 201 Created
{
  "userId": "65d1a2b3c4e5f6789012345",
  "username": "bookworm42",
  "message": "User registered successfully"
}

# Step 2: Login to get JWT
POST http://localhost:8080/api/v1/auth/login
Content-Type: application/json

{
  "username": "bookworm42",
  "password": "SecurePass123!"
}

# Response: 200 OK
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "userId": "65d1a2b3c4e5f6789012345",
  "username": "bookworm42",
  "role": "USER"
}

# Step 3: Create a review (using JWT from step 2)
POST http://localhost:8080/api/v1/me/reviews
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json

{
  "bookId": "698db76b4a1f5c8fa49331a8",
  "rating": 95,
  "text": "An absolute masterpiece! Tolkien's world-building is
    ...",
  "summary": "Amazing fantasy classic"
}

# Response: 201 Created
{
  "id": "65d1a2b3c4e5f6789012999",
  "bookId": "698db76b4a1f5c8fa49331a8",
  "bookTitle": "The Fellowship of the Ring",
  "rating": 95,
  "text": "An absolute masterpiece! Tolkien's...",
  "summary": "Amazing fantasy classic",
  "createdAt": "2026-02-13T16:30:00Z",
  "likesCount": 0
}

```

7.4.2 Example 2: Analytics Query

```
# Get current trending books (top 10)
GET http://localhost:8080/api/v1/analytics/rankings/trendingbooks

# Response: 200 OK
[
  {
    "id": "698db76b4a1f5c8fa49331a8",
    "title": "The Fellowship of the Ring",
    "author": "J. R. R. Tolkien",
    "averageRating": 92.5,
    "trendScore": 1450,
    "recentReviewsCount": 45
  },
  {
    "id": "698db76c4a1f5c8fa4936c8e",
    "title": "1984",
    "author": "George Orwell",
    "averageRating": 89.3,
    "trendScore": 1320,
    "recentReviewsCount": 38
  }
]
```

Listing 7.3: Trending Books Analytics API

7.5 Error Handling

The API follows RFC 7807 (Problem Details for HTTP APIs) for consistent error responses:

```
1 {
2   "timestamp": "2026-02-13T16:35:22.123Z",
3   "status": 404,
4   "error": "Not Found",
5   "message": "Book with ID 'invalid-id' not found",
6   "path": "/api/v1/books/invalid-id"
7 }
```

Listing 7.4: Example Error Response

7.6 Security

- **JWT Expiration:** Tokens expire after 24 hours
- **Password Policy:** Minimum 8 characters, encrypted with BCrypt (strength 10)
- **Input Validation:** All requests validated using Jakarta Bean Validation (JSR 380)
- **CORS:** Configured for specified frontend origins

Appendix A

AI Tools Usage

This appendix documents the use of Large Language Models (LLMs) and AI tools throughout the development of the BookSphere project.

A.1 Purpose

AI tools, specifically **GitHub Copilot** (powered by Claude Sonnet 4.5) and **Gemini Pro 3**, were strategically employed across two primary domains:

Code Implementation

- **Dataset Preprocessing:** Used to elaborate raw data to fit the databases design.
- **Spring Boot Annotations:** Suggestions for `@Retryable`, `@Async`, and `@Cacheable` usage patterns to handle transient failures and improve responsiveness.
- **Error Handling:** Implementation of RFC 7807 Problem Details standard for RESTful API error responses with consistent HTTP status codes.
- **Testing:** Java unit tests, performance tests with Gatling Scala syntax for scenario injection and Postman Collections for End-to-End tests.

Documentation Generation

AI tools were utilized to assist with the formatting of complex tables, algorithm blocks, and syntax highlighting configurations for JSON, Java, and Cypher code listings. Key contributions include:

- **API Endpoint Tables:** Auto-generation of comprehensive endpoint tables (52 total endpoints) categorized by authentication level (Open, Registered, Admin).
- **Code Comment Refinement:** Improvement of JavaDoc comments for service methods, particularly explaining retry logic, transaction boundaries, and eventual consistency patterns.

- **Performance Metrics Formatting:** Structuring Gatling test results into readable tables with response time percentiles, throughput, and success rates.

A.2 Prompts and Tasks Performed

Below are representative examples of AI interactions categorized by development phase:

Implementation Phase

- **Prompt (Copilot inline):** "Retry logic for Neo4j connection failures in follow operation"

Generated code:

```
@Retryable(
    retryFor = {RuntimeException.class},
    maxAttempts = 3,
    backoff = @Backoff(delay = 1000, multiplier = 2)
)
public void followUser(String followerId, String followedId) {
    // Method body
}
```

Applied to: 20+ service methods across FollowService, LikeService, BookshelfService.

- **Prompt:** "Create a Gatling scenario with ramp-up of 20 users over 10s, then constant 10 req/s for 30s, with assertions for 95% success rate and <2s mean response time."

Usage: Template for `BasicLoadTest.scala`, adapted for 5 different test types (Stress, Spike, Endurance, DatabaseIntensive).

Documentation Phase

- **Prompt:** "Generate a LaTeX table with 9 rows for API endpoints including columns: Method, Path, Auth Required, Description."

Usage: Created Tables 7.1, 7.2, 7.3 in Section 7.3.

- **Prompt:** "Format this Gatling JSON output as a LaTeX itemize list with mean/p95/p99 response times in milliseconds."

Usage: Metrics presentation in Section 6.3.

A.3 Critical Evaluation

While AI tools significantly accelerated development, multiple critical modifications were necessary:

A.3.1 Code Corrections and Architectural Refinements

To address architectural pitfalls and specific framework constraints that the LLM overlooked.

1. **Neo4j Entity Mapping & JVM Heap Saturation:** The initial code generated by the AI included full bidirectional relationships within the Spring Data Neo4j entity classes (e.g., the `Book` node containing a `List<Review>` field, and `Review` containing `User`, which linked back to other books). Due to the highly interconnected nature of the graph, retrieving a single entity triggered a cascade of eager fetch operations ("subgraph explosion"). This caused the application to attempt loading vast portions of the graph into memory, rapidly **exhausting the JVM Heap space** and crashing the application.

Correction: We completely redesigned the Node entities to be lightweight, removing complex relationship fields. Relationships are now handled exclusively via specific Cypher queries and Projections, fetching only the required data depth.

2. **Asynchronous Execution (@Async) & Thread Management:** The AI initially implemented `@Async` methods within the same Service class where they were invoked (e.g., calling `this.updateStats()` inside `ReviewService`). In the Spring Framework, this "self-invocation" pattern prevents the application from spawning a **separate thread** for the task. Consequently, the operations were executing synchronously, blocking the main request thread and defeating the purpose of asynchronous processing. Additionally, the AI often generated these methods with `private` visibility, which is incompatible with the framework's async management. *Correction:* We refactored the architecture by extracting all asynchronous logic into dedicated components (e.g., `AsyncReviewTasks`), ensuring that these operations are correctly executed as **parallel tasks** in the background.

A.3.2 Validation Against Course Requirements

All AI-generated content was cross-validated against:

- Spring Framework reference (transaction boundaries, async execution semantics)
- Course Lectures on Database Design and good practice

A.4 Ethical Considerations

All AI-generated code and documentation:

- Was reviewed, tested, and modified by team members before submission
- Was validated against course learning objectives to ensure understanding of underlying concepts

This project demonstrates responsible AI usage: leveraging automation for productivity while maintaining critical thinking, technical validation, and personal accountability for all submitted work.

Bibliography

- [1] Mohamed Bakhet. *Amazon Books Reviews Dataset*. Kaggle. 2021. URL: <https://www.kaggle.com/datasets/mohamedbakhet/amazon-books-reviews>.
- [2] Ruchi Bhatia. *Book-Crossing Dataset*. Kaggle. 2021. URL: <https://www.kaggle.com/datasets/ruchi798/bookcrossing-dataset>.
- [3] *Large-Scale and Multi-Structured Database project: bookSphere*. Version 1.0. 2026. URL: <https://github.com/matteobilliciani/Large-Scale-and-Multi-Structured-Database-project-bookSphere>.